



数字逻辑与处理器基础

Fundamentals of Digital Logic and Processor



第四讲 时序逻辑 (1/2)

李学清

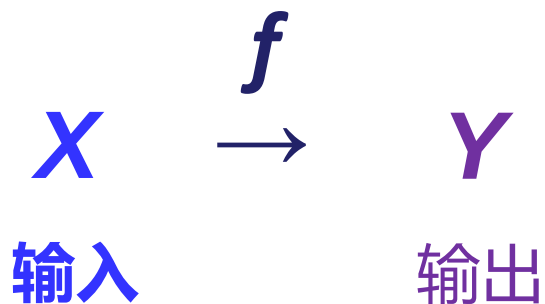
#腾讯会议：453-1488-6548

助教：曾令安*/唐文骏/陈仲豪

*本节内容负责答疑助教

2024年9月30日

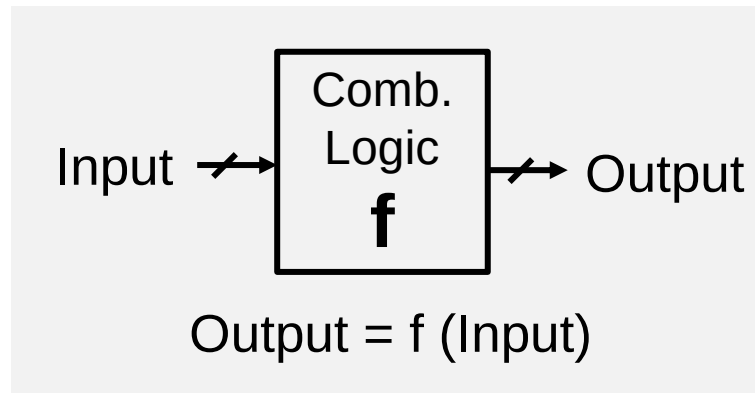
时序逻辑的作用



周	日期	暂定主要内容
1	9/9	绪论
2	9/21	布尔代数 (中秋节调休)
3	9/23	组合逻辑
4	9/30	时序逻辑
5	10/7	国庆节放假
6	10/14	时序逻辑
7	10/21	计算机指令系统
8	待定	期中考试 (TBD)
9	11/4	计算机指令系统
10	11/11	微处理器
11	11/18	微处理器
12	11/25	流水线
13	12/2	流水线
13	12/7	流水线/存储器 (调课)
15	12/16	存储器
16	12/23	存储器/IO与总线/总结

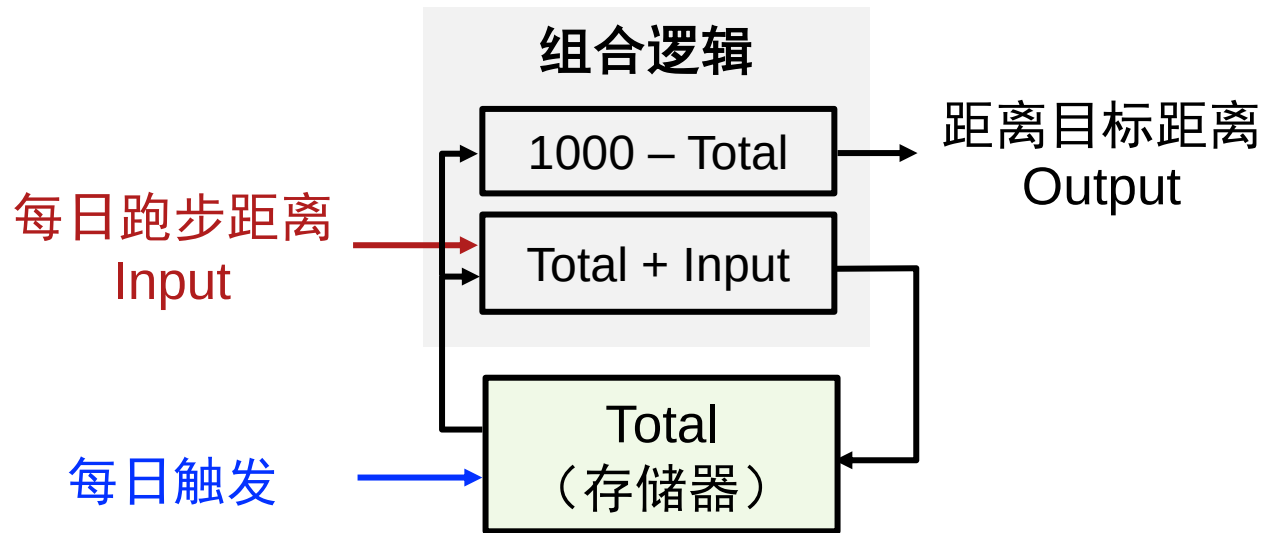
问题

- 上次课程：你需要一个64-bit加法器



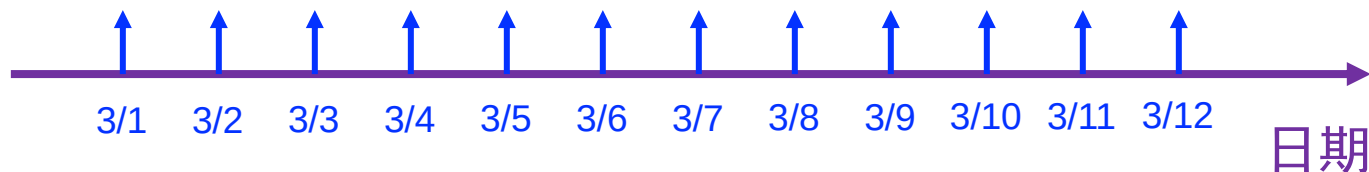
- 本次课程
 - 如果需要把历史输入也加进来？
 - 或者，需要保存一些数据将来计算使用？
 - 例如：记录阳光长跑项目中“已经跑完的距离”以及“还差多少距离才达到1000km”

问题



Output = f(Input, Total States)

Total States $\leq$ g(Input, Previous States)



本节课学习目标

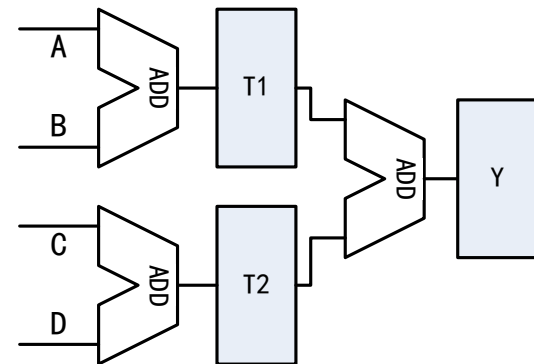
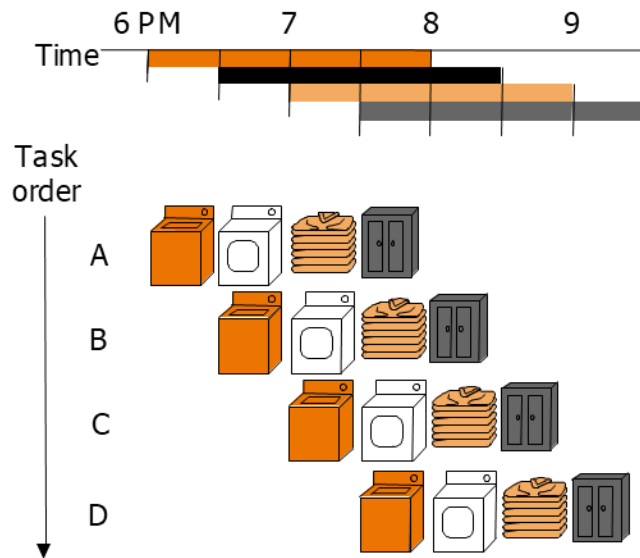
- 理解时序逻辑概念、理论、指标、非理想因素
 - 对比组合逻辑
- 分析和设计时序逻辑电路
 - 优化
 - 使用
- 树立过程同步化处理意识

本节课目录

- 概念：时钟，状态和FSM
- 时序逻辑电路：锁存器
- 时序逻辑电路：DFF
- 分析，设计，举例

更多原因 – 为什么使用时序逻辑？

- 过去的输入很重要
- 分步完成计算任务
- 提升性能 (吞吐量)

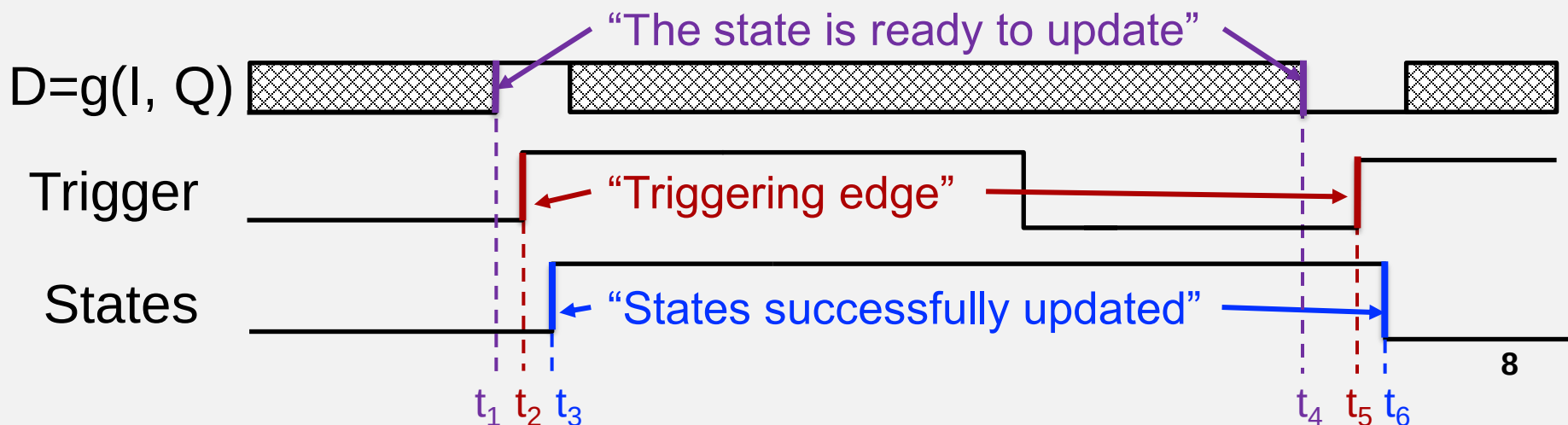
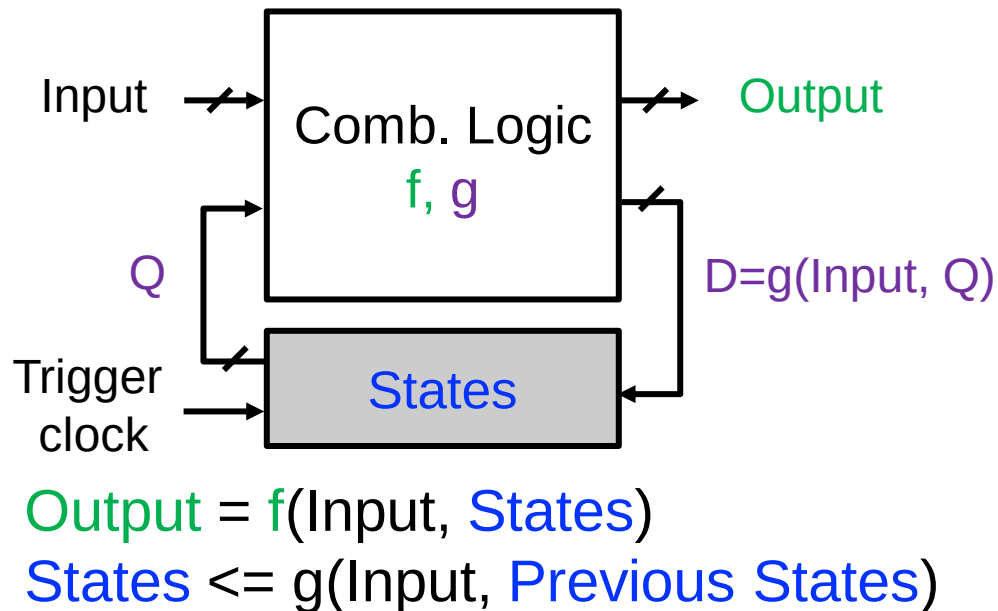


同步节拍以实现更高的吞吐量

从组合逻辑到时序逻辑

本节课需要思考的问题：

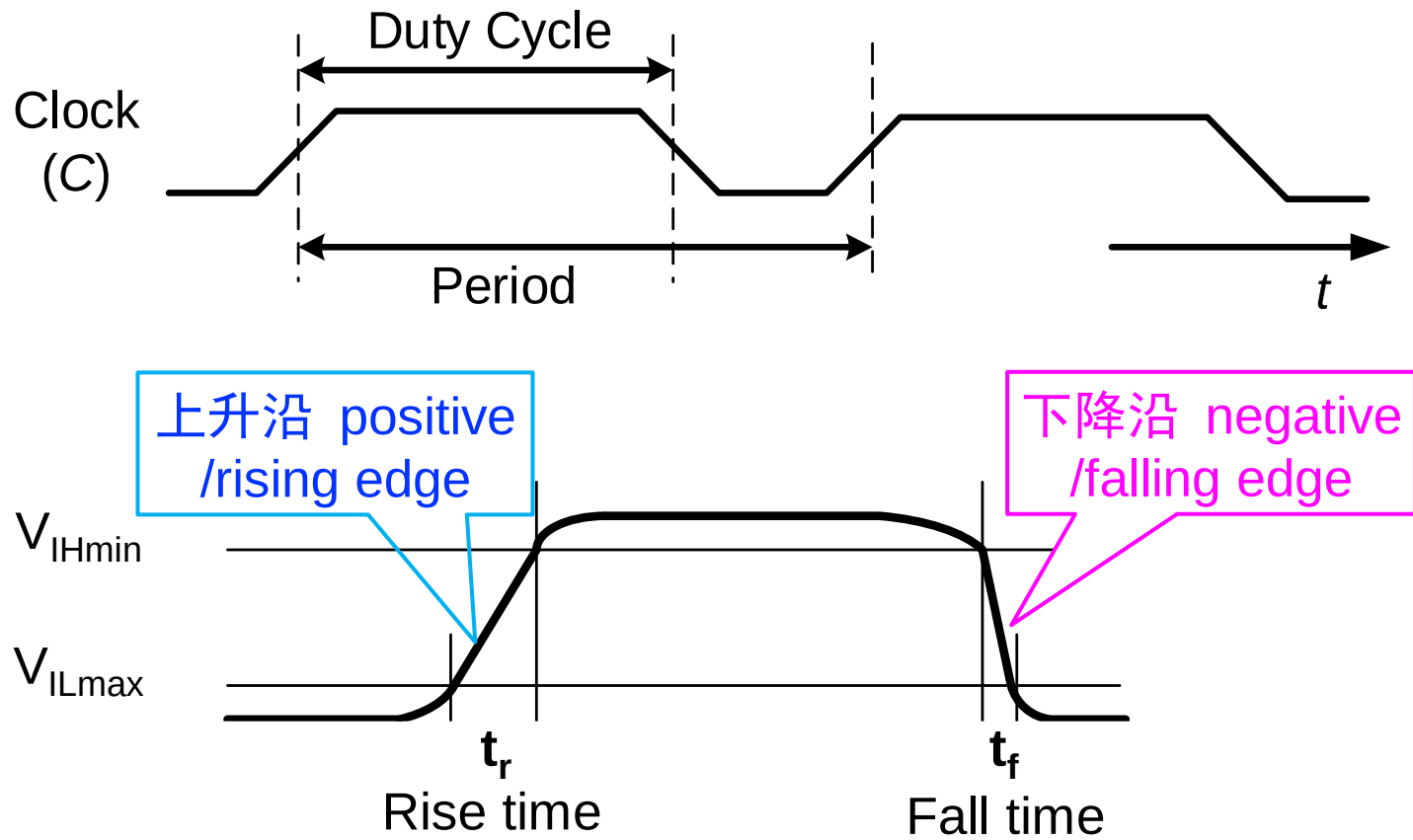
1. 如何提供触发控制
2. 如何存储状态
3. 如何保证时序



触发与节拍信号：时钟

- 参数

- 周期=1/频率, 占空比, 上升时间, 下降时间
- Period=1/frequency, duty-cycle, rise time, fall time



时钟生成：环形振荡器

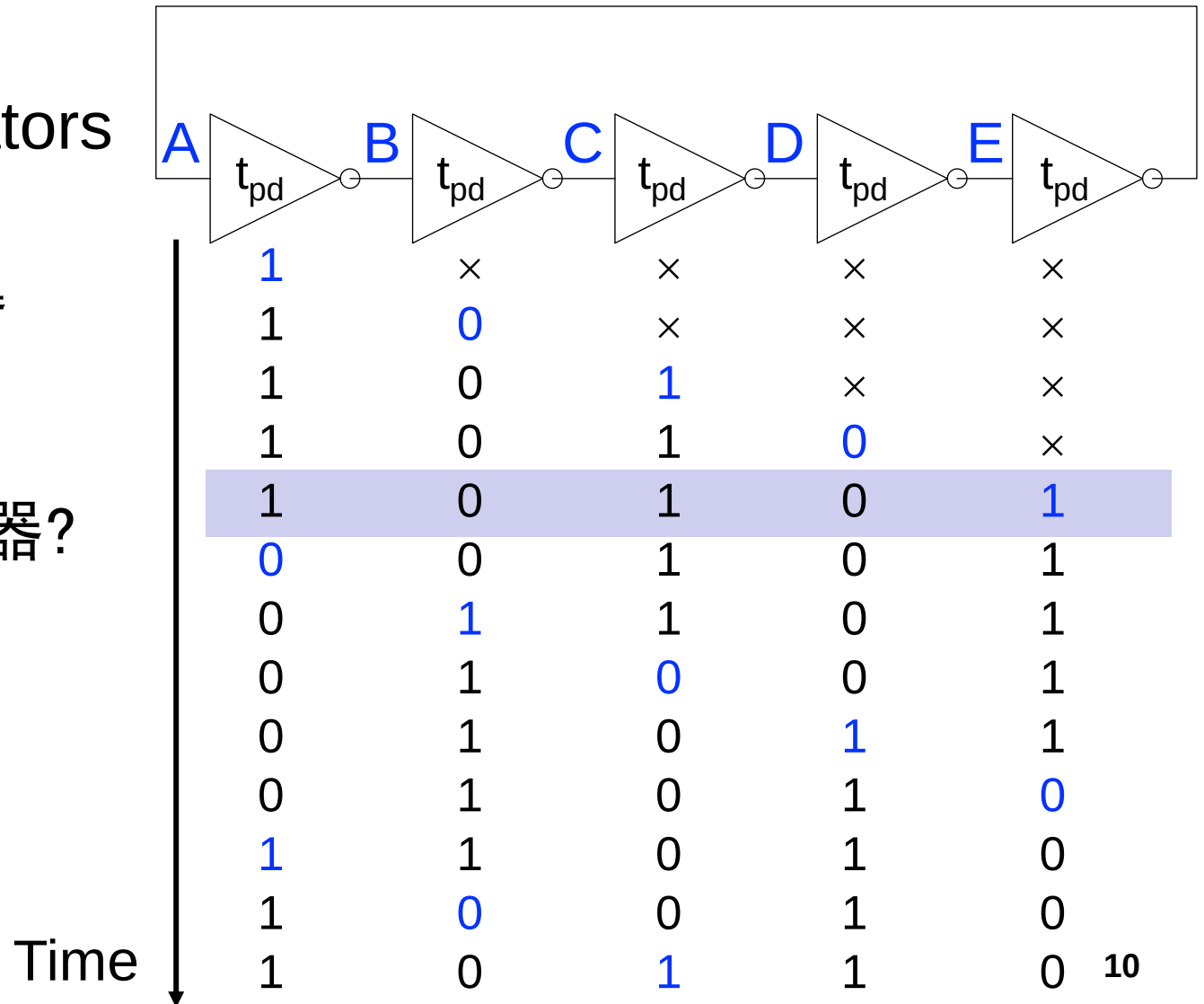
- Ring Oscillators (RO)

- 环形振荡器

- 周期?

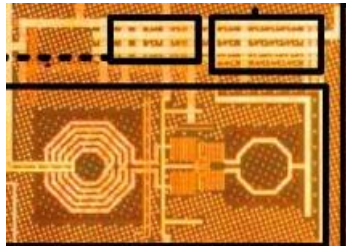
- 多少个反相器?

- 奇数个(≥ 3)

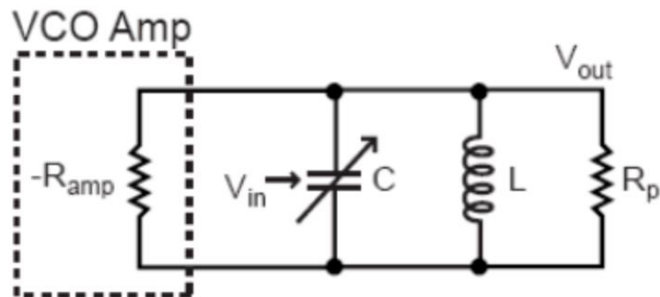


其他时钟信号发生器

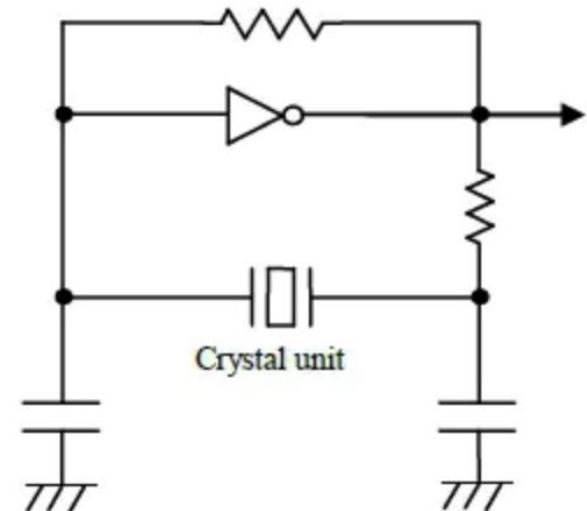
- LC oscillator (LC 振荡器)
- Crystal oscillator (晶振)



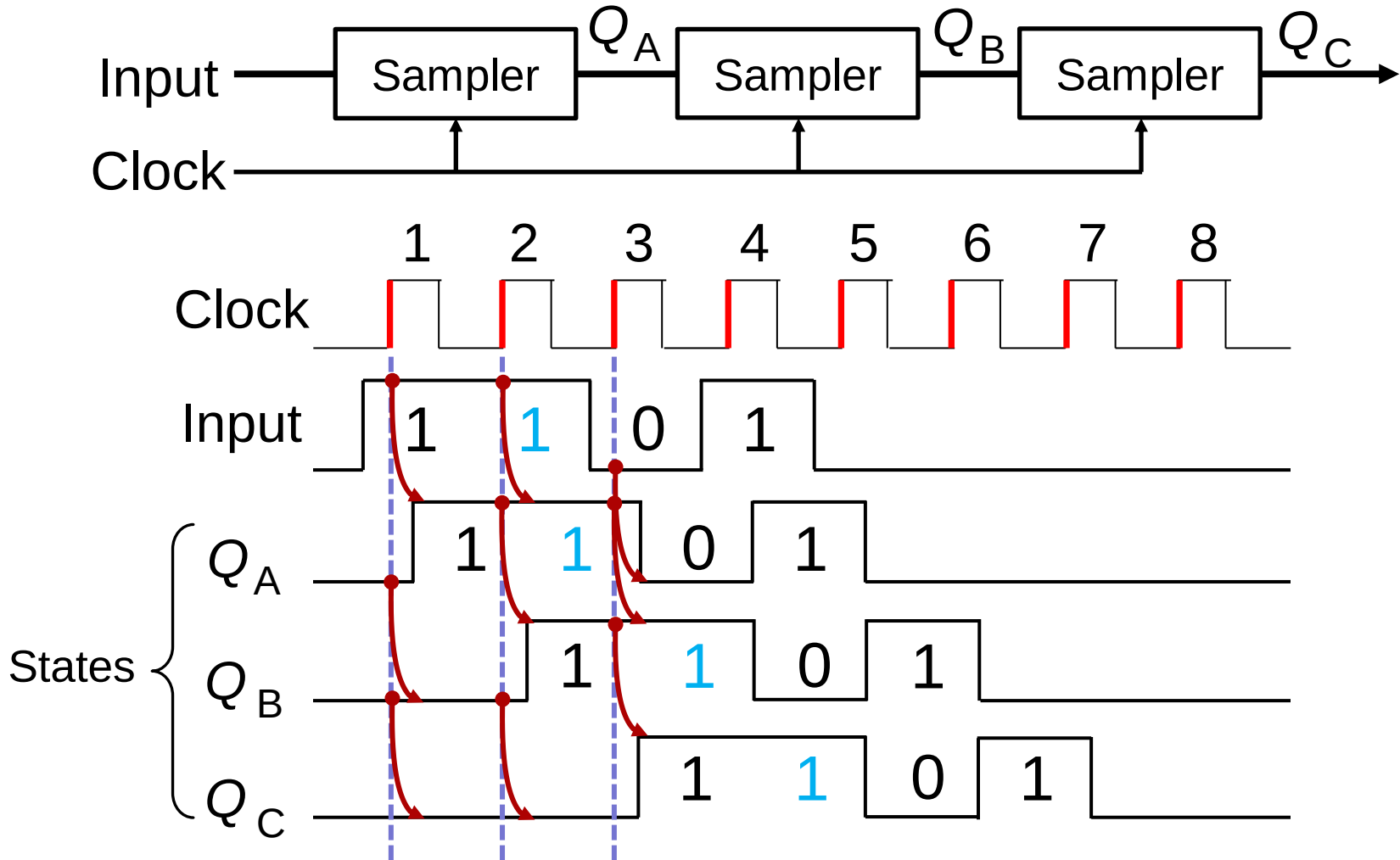
LC oscillator



Crystal oscillator



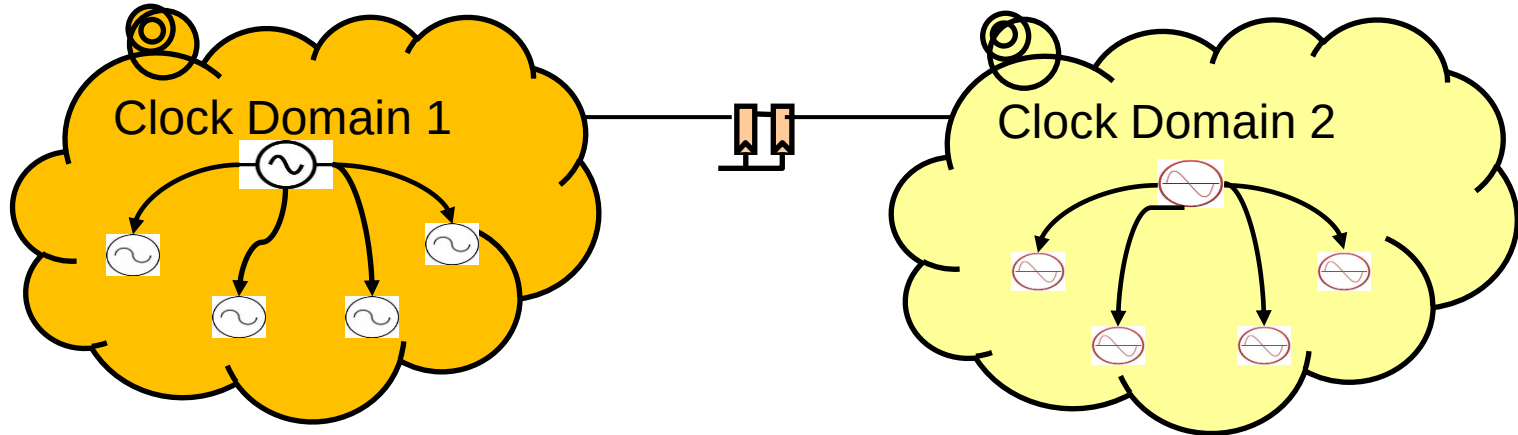
时钟触发：在边缘采样



采样到输出存在一些延时: Clock-Q delay

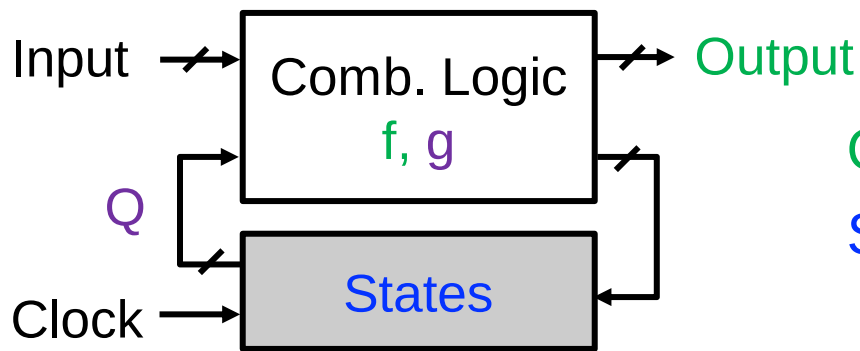
时钟域 (Clock domain)

- **同步**: 使信号事件与时钟节拍一致
- **时钟域**: 所有同步于同一个时钟信号的信号集合



“状态” 的定义与FSM

- 一个系统的状态(State)应包含所有需要的信息，可以有冗余
 - 依据特定的输入可以预测未来所有的行为
 - 状态的集合
- 有限状态机(Finite State Machine, FSM)
 - 输入、输出和状态的数目都是有限的
 - 描述一个FSM，6大元素
 - State (initial: S_0), input, output, f , g

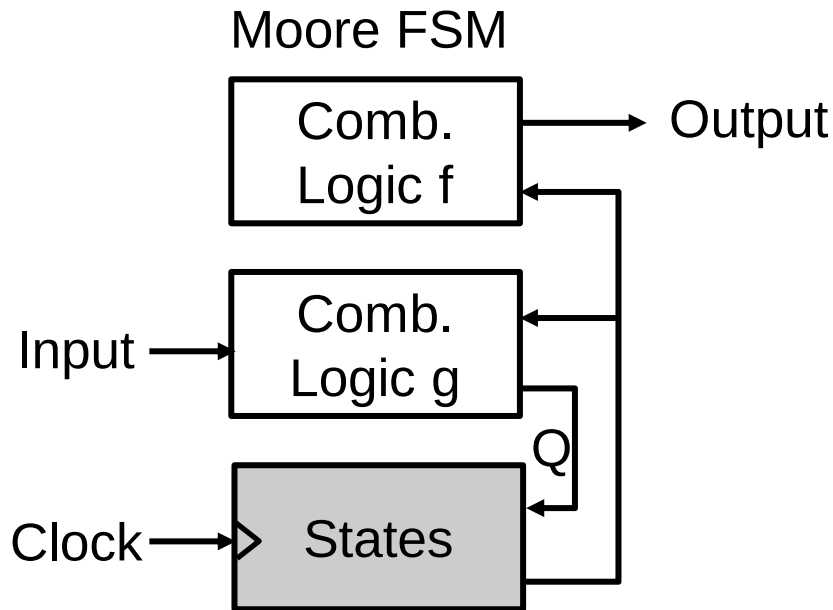


$\text{Output} = f(\text{Input}, \text{States})$

$\text{States} \leftarrow g(\text{Input}, \text{Previous States})$

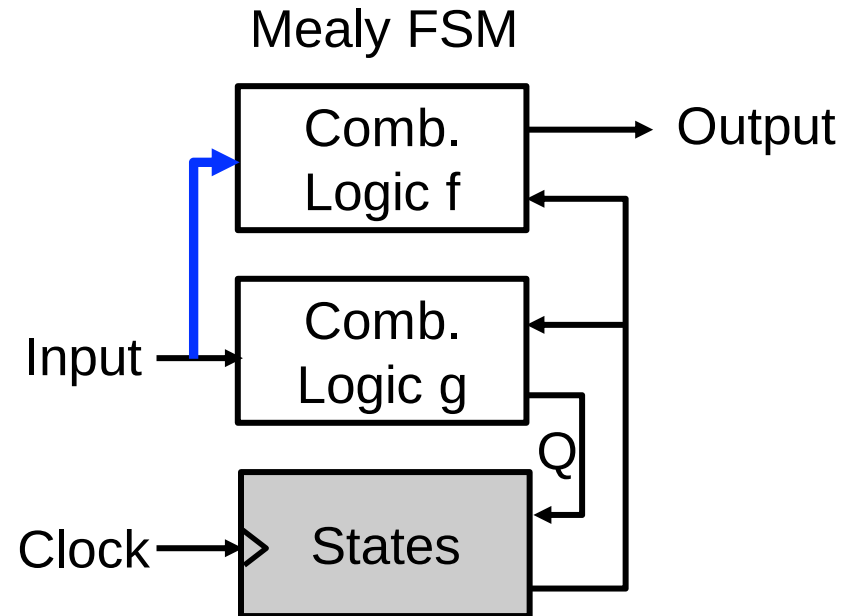
FSM 类型: Mealy vs. Moore

- 摩尔机(Moore FSM): 输出只与状态有关
- 米利机(Mealy FSM): 输出与状态和输入有关



Output = f(States)

States <= g(Input, State_old)



Output = f(Input, States)

States <= g(Input, State_old) ⁷

FSM描述: 状态转换表(State transition table)

- 和卡诺图相似
 - 描述下一个状态和输出
 - 摩尔机和米利机的区别
- 2D vs 1D table

Input State	I_1	I_2	Output
S_1	S_a	S_c	O_b
S_2	S_e	S_g	O_f
...	...	...	...
S_m	S_k	S_m	O_l

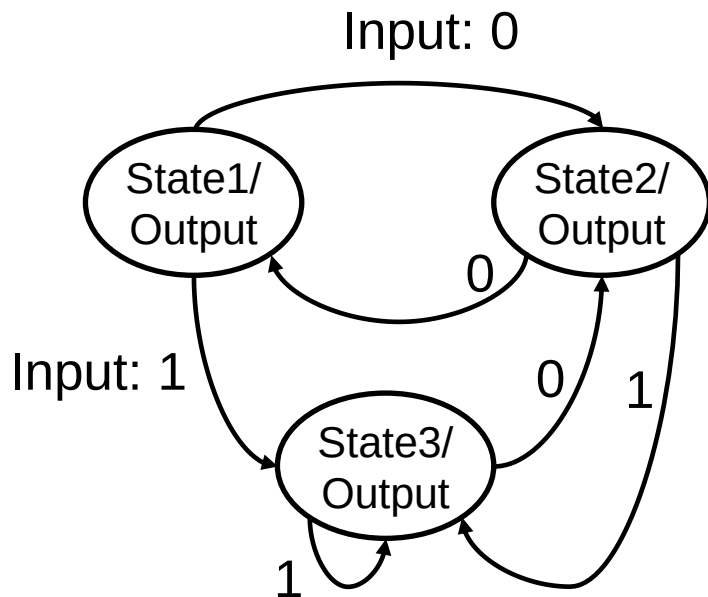
Moore FSM

Input State	I_1	I_2
S_1	S_a/O_b	S_c/O_d
S_2	S_e/O_f	S_g/O_h
...	...	...
S_m	S_k/O_l	S_m/O_n

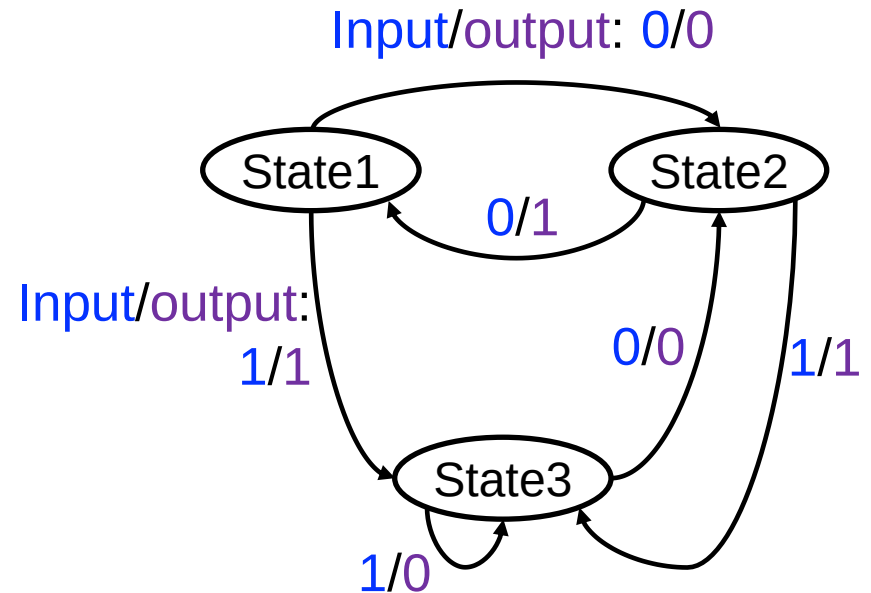
Mealy FSM

FSM描述: 状态图(State diagram)

- 圆圈表示状态，箭头表示转移方式
 - 摩尔机和米利机的区别



Moore example
(1-b input, 3-state)

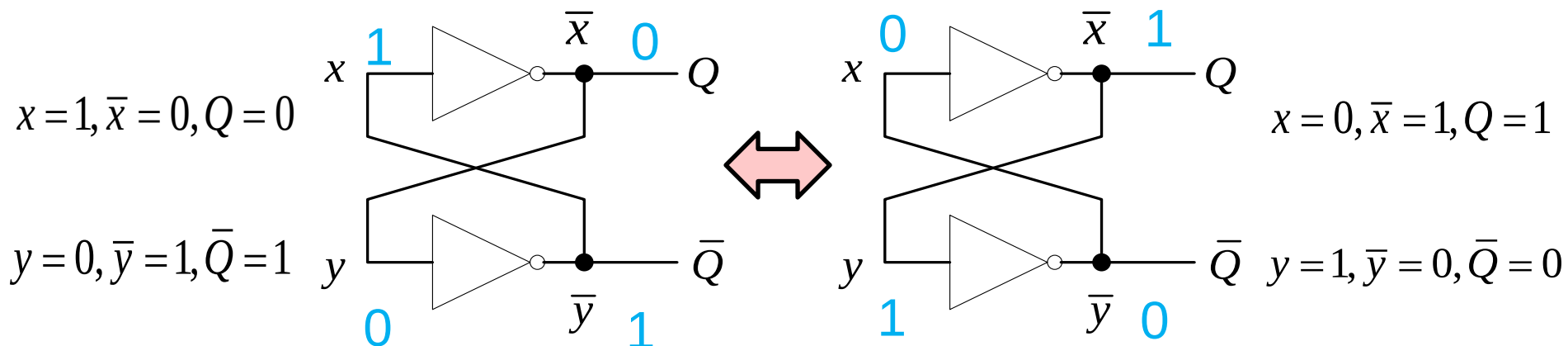


Mealy example
(1-b input, 1-b output, 3-state)

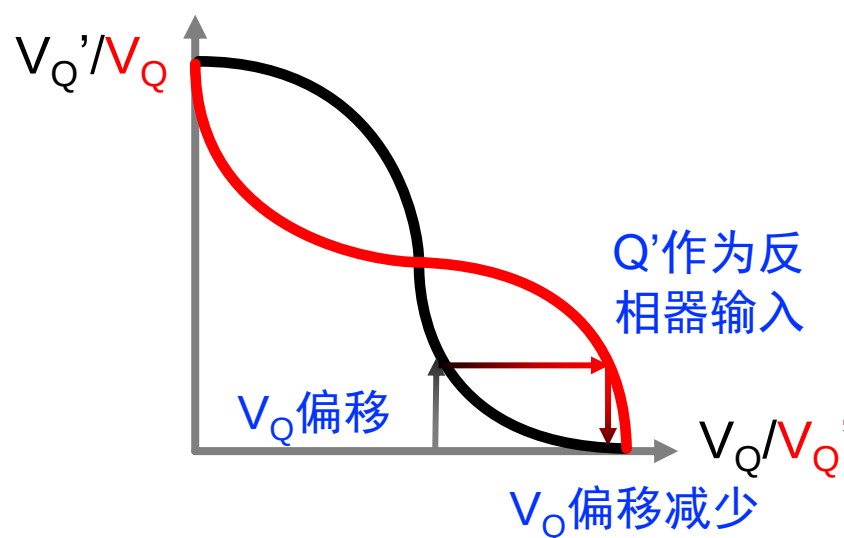
本节课目录

- 概念：时钟，状态和FSM
- 时序逻辑电路：锁存器
- 时序逻辑电路：DFF
- 分析，设计，举例

存储1bit状态的双稳态单元

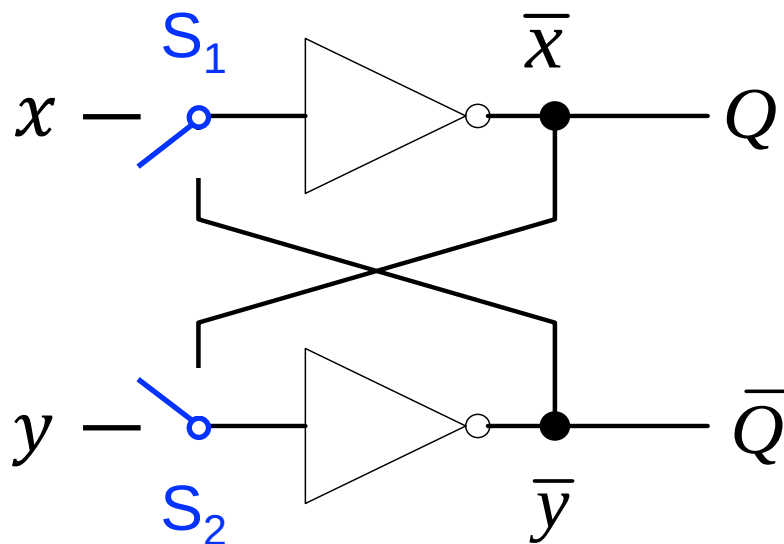


为什么叫双稳态单元？

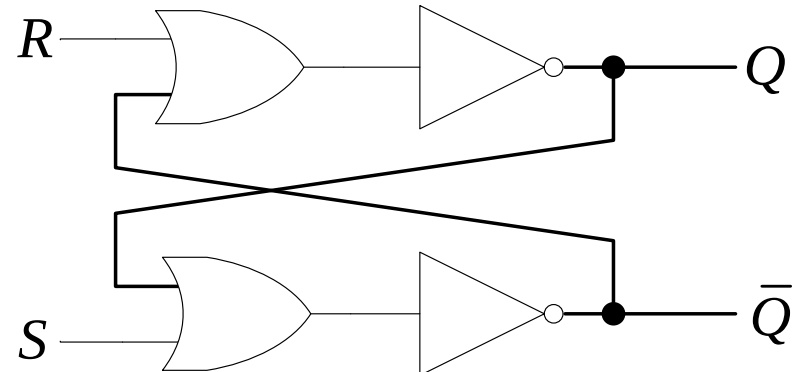
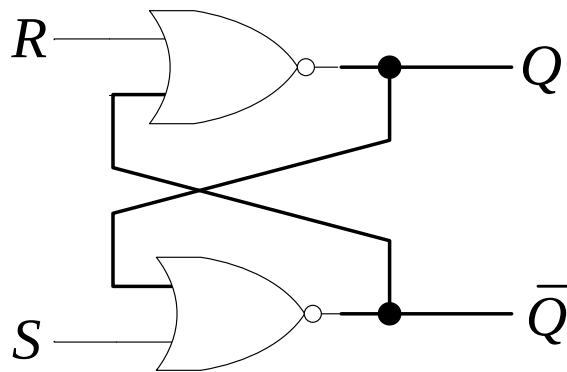


如何使用这个电路？

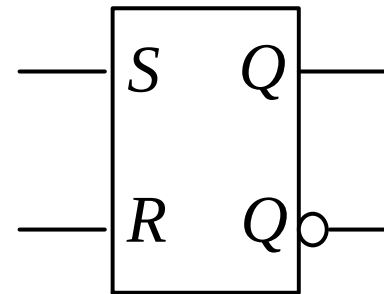
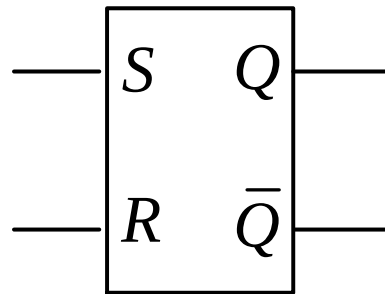
- 如何预设初始状态？
- 如何改变这个状态？



SR锁存器(SR latch)

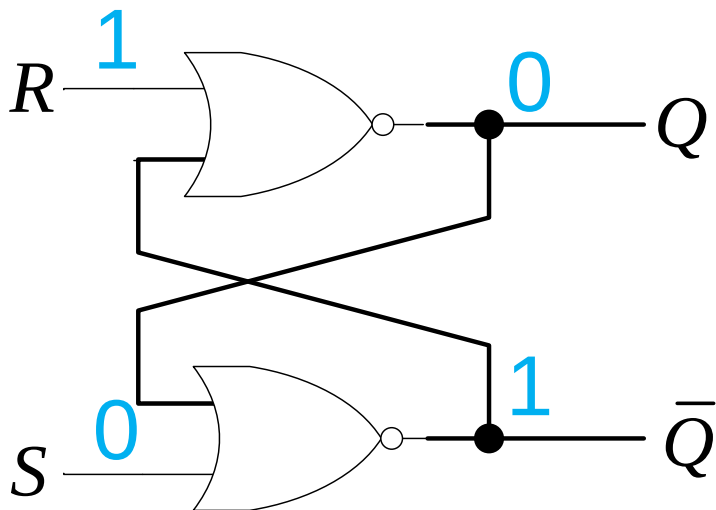


电路图



逻辑符号

SR锁存器：复位



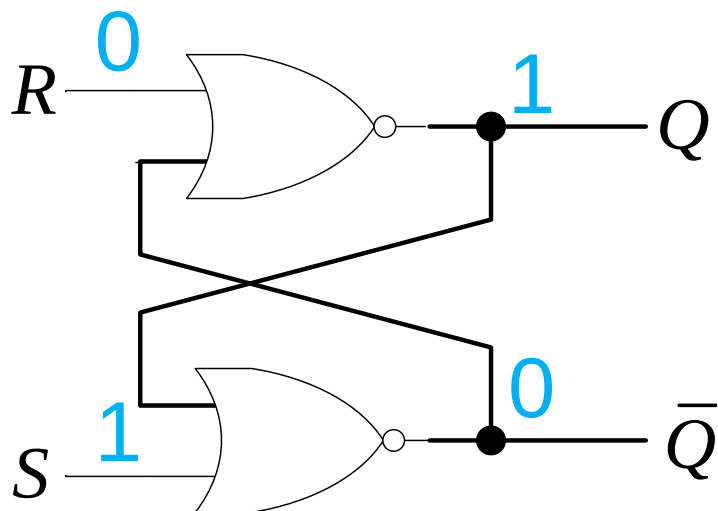
R: 复位端

功能表

R	S	Q^+	$\bar{Q}^+$
1	0	0	1

```
module SR_latch(R, S, Q, Q_bar);  
  input R,S;  
  output Q,Q_bar;  
  nor n1(Q, R, Q_bar);  
  nor n2(Q_bar, S, Q);  
endmodule
```

SR锁存器：置位



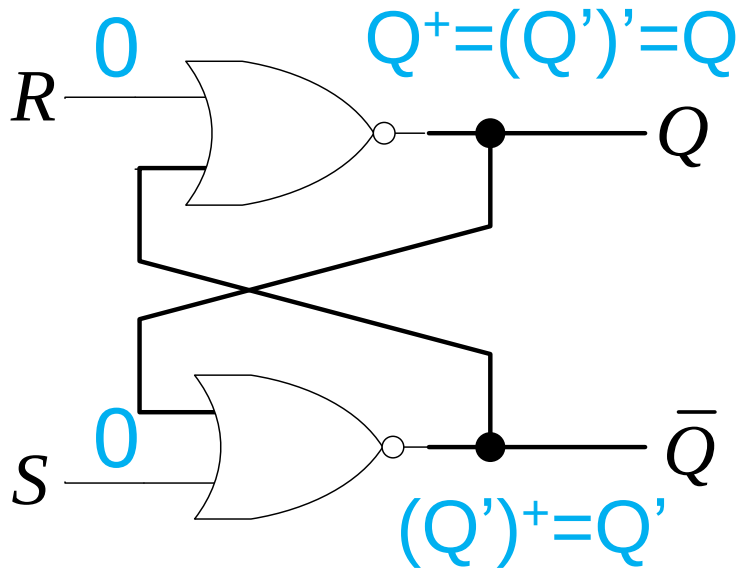
S: 置位端

功能表

R	S	Q^+	$\bar{Q}^+$
1	0	0	1
0	1	1	0

```
module SR_latch(R, S, Q, Q_bar);  
  input R,S;  
  output Q,Q_bar;  
  nor n1(Q, R, Q_bar);  
  nor n2(Q_bar, S, Q);  
endmodule
```

SR锁存器: 保持



R: 复位端

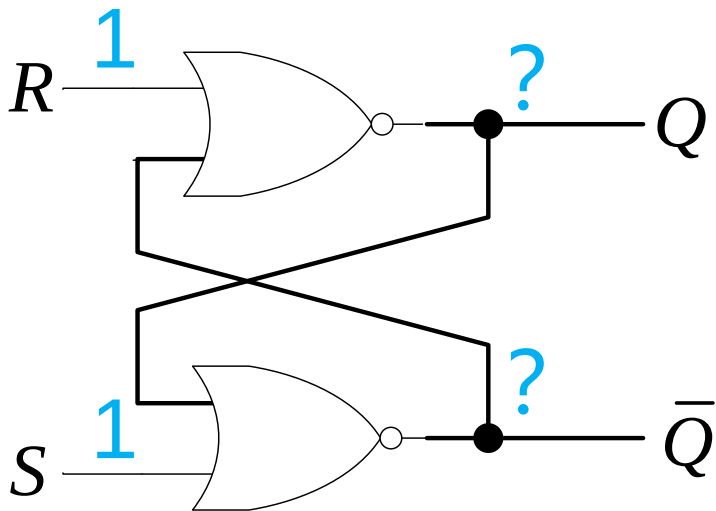
S: 置位端

功能表

R	S	Q^+	$\bar{Q}^+$
1	0	0	1
0	1	1	0
0	0	Q	Q'

```
module SR_latch(R, S, Q, Q_bar);  
  input R,S;  
  output Q,Q_bar;  
  nor n1(Q, R, Q_bar);  
  nor n2(Q_bar, S, Q);  
endmodule
```


SR锁存器: 如果SR=1?



R: 复位端

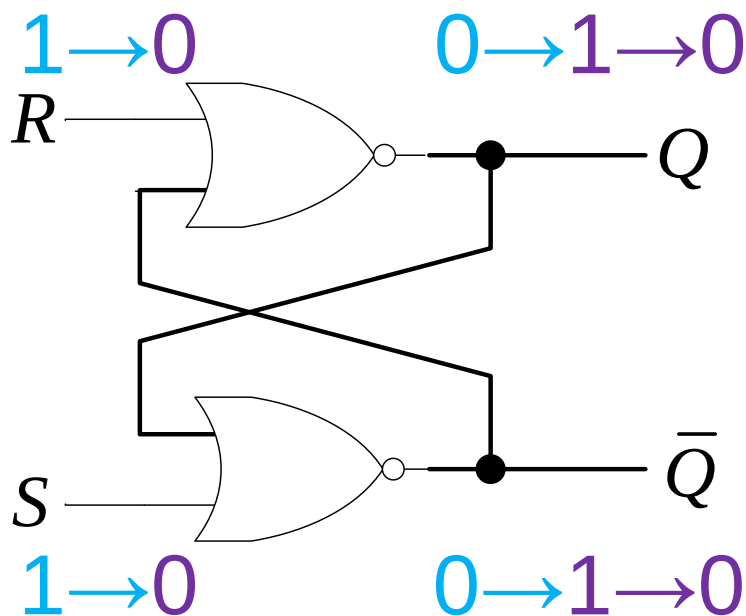
S: 置位端

功能表

R	S	Q^+	$\bar{Q}^+$
1	0	0	1
0	1	1	0
0	0	Q	Q'
1	1	?	?

- 当 $S=R=1$ 时, $Q=Q'=0$ (稳态)

SR锁存器: 为什么要求SR=0?



功能表

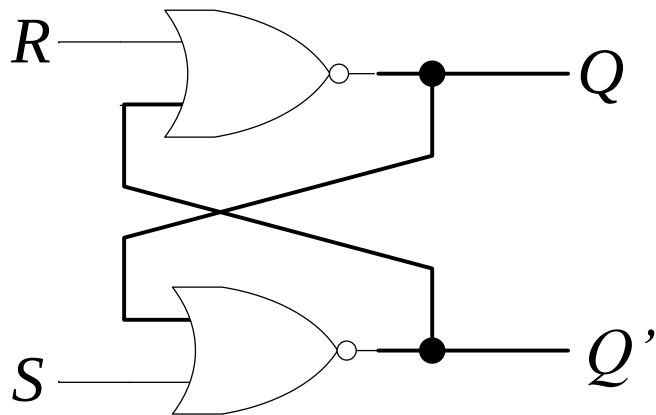
$$Q^+ = S + R'Q \quad (SR=0)$$

- 当 $S=R=1$ 时, $Q=Q'=0$ (稳态)
- 但是, 当 S 和 R 之后同时变为 0, 输出不稳定 (需要避免)

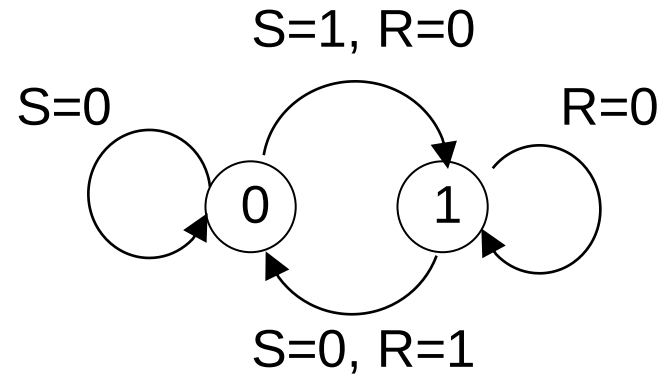
R	S	Q^+	$\bar{Q}^+$
1	0	0	1
0	1	1	0
0	0	Q	Q'
1	1	Not Allowed	

Q/Q' 为 1/0 或 0/1:
存储 1bit 信息!

SR锁存器的状态图



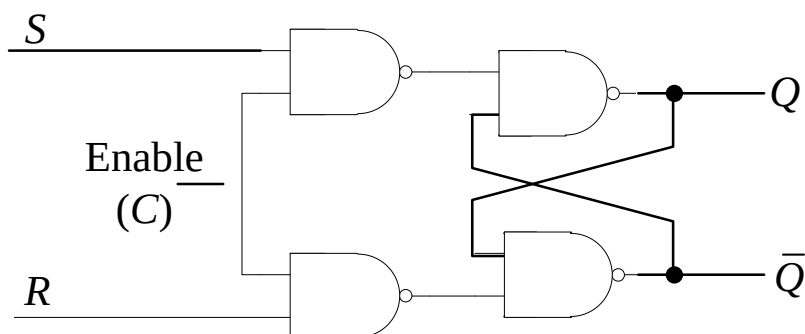
$$Q^+ = S + R'Q \quad (SR=0)$$



Q/Q' 为 1/0 或 0/1:
存储1bit信息!

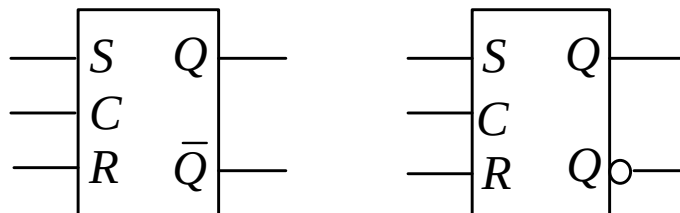
门控SR锁存器

仅当使能C=1时才可以改变状态



$$Q^+ = C'Q + C(S + R'Q), \text{ SR}=0$$

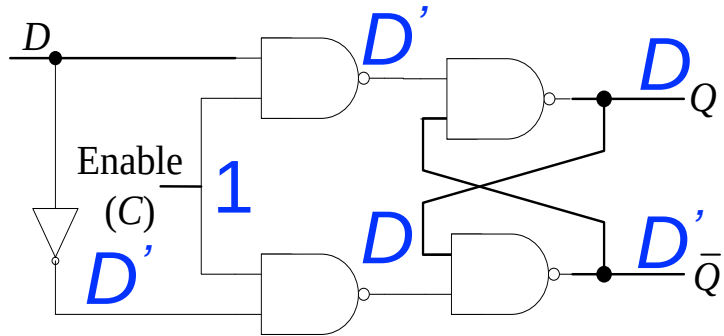
$$Q^+ = S + R'Q, \text{ C下降沿时锁定}$$



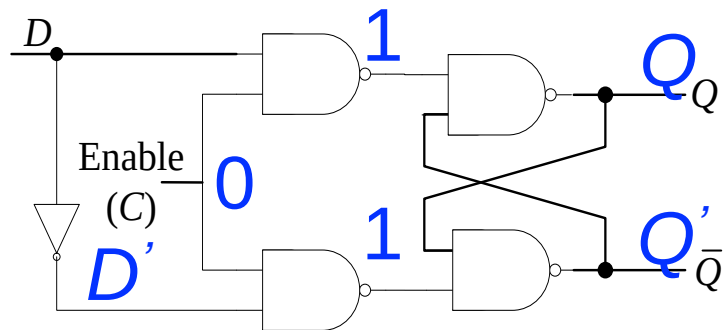
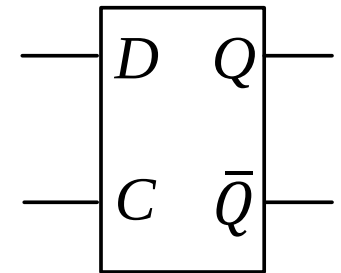
```
module SR_E_latch(R, S, C, Q, Q_bar);
    input R, S, C;
    output Q, Q_bar;
    wire d1, d2;
    nand n1(d1, S, C);
    nand n2(d2, C, R);
    nand n3(Q, d1, Q_bar);
    nand n4(Q_bar, d2, Q);
endmodule
```

S	R	C	Q^+	$\bar{Q}^+$
1	0	1	1	0
0	1	1	0	1
0	0	1	Q	$\bar{Q}$
1	1	1	$\times$	$\times$
$\times$	$\times$	0	Q	$\bar{Q}$

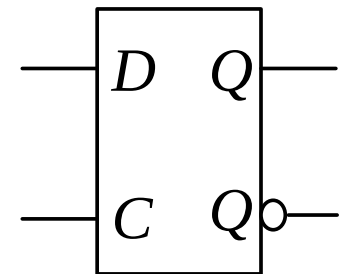
(门控) D锁存器



D	C	Q^+	$\bar{Q}^+$
0	1	0	1
1	1	1	0
$\times$	0	Q	$\bar{Q}$

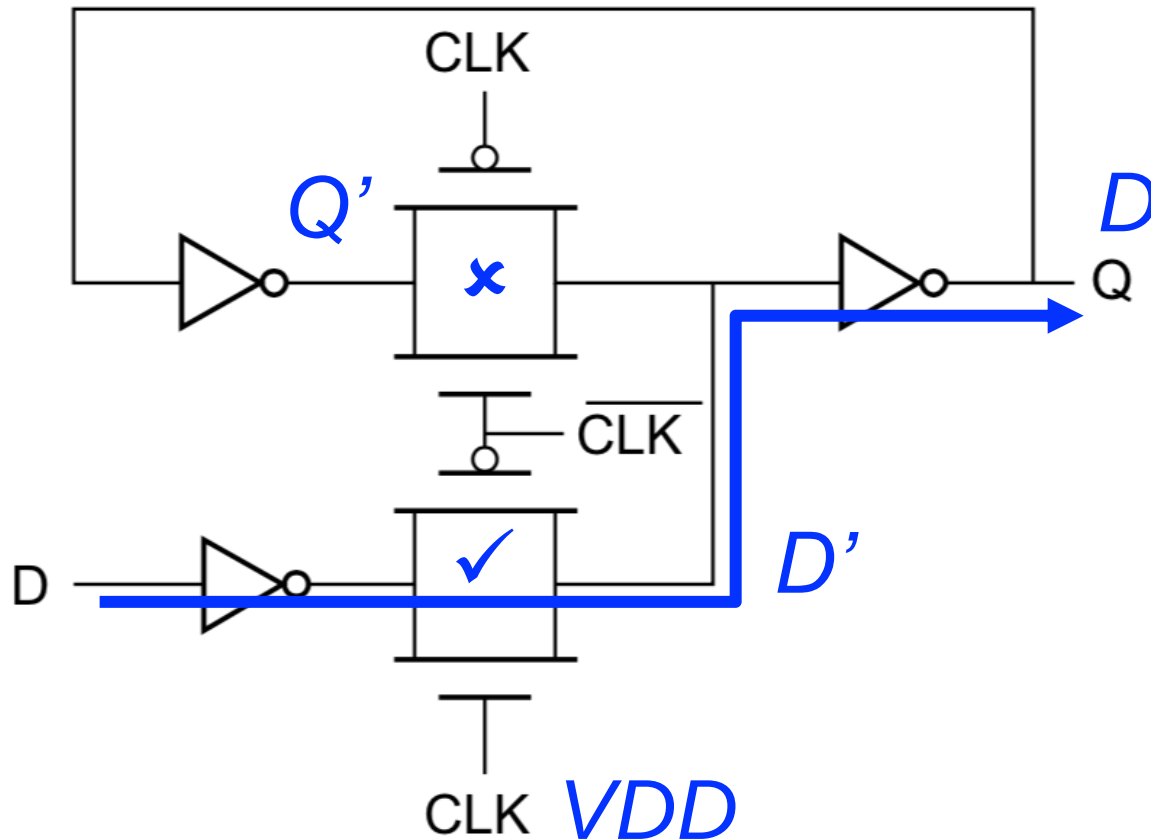


D	C	Q^+	$\bar{Q}^+$
0	1	0	1
1	1	1	0
$\times$	0	Q	$\bar{Q}$



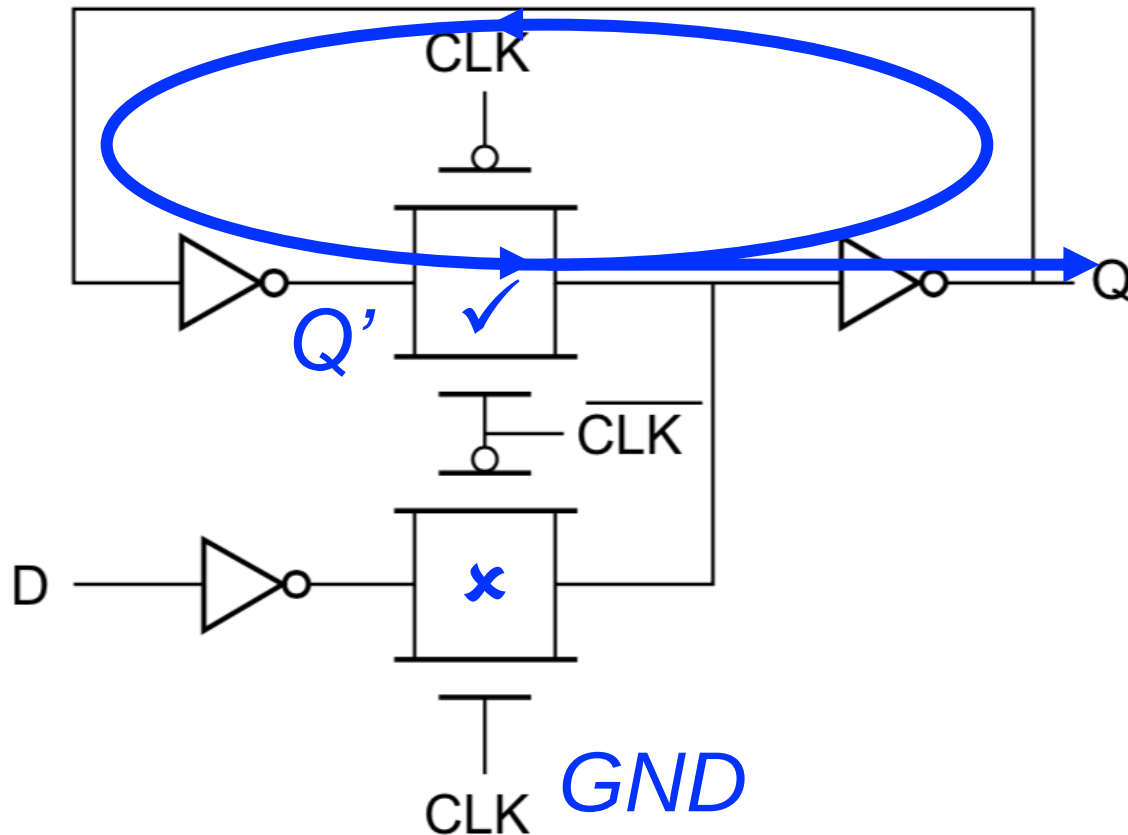
$Q^+ = D$: C为高时跟踪D, 下降沿锁定

(门控) D锁存器：另一种实现方法



$Q^+ = D$: 下降沿锁定, CLK为高时Q跟踪D(透明)

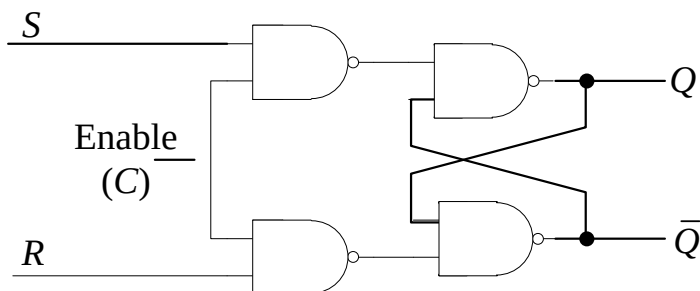
(门控) D锁存器：另一种实现方法



$Q^+ = D$: 下降沿锁定, CLK 为低时 Q 和 Q' 形成闭环(锁定)

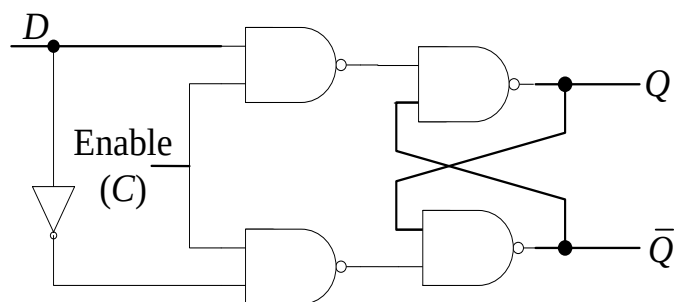
时序差异：门控SR锁存器 vs D锁存器

- 都是C下降沿锁定
- 门控D锁存器的锁定输出只与下降沿的输入有关
- 门控SR锁存器：S和R端都为低也可以锁定输入



S	R	C	Q^+	$\bar{Q}^+$
1	0	1	1	0
0	1	1	0	1
0	0	1	Q	$\bar{Q}$
1	1	1	$\times$	$\times$
$\times$	$\times$	0	Q	$\bar{Q}$

$$Q^+ = S + R'Q, (SR=0)$$



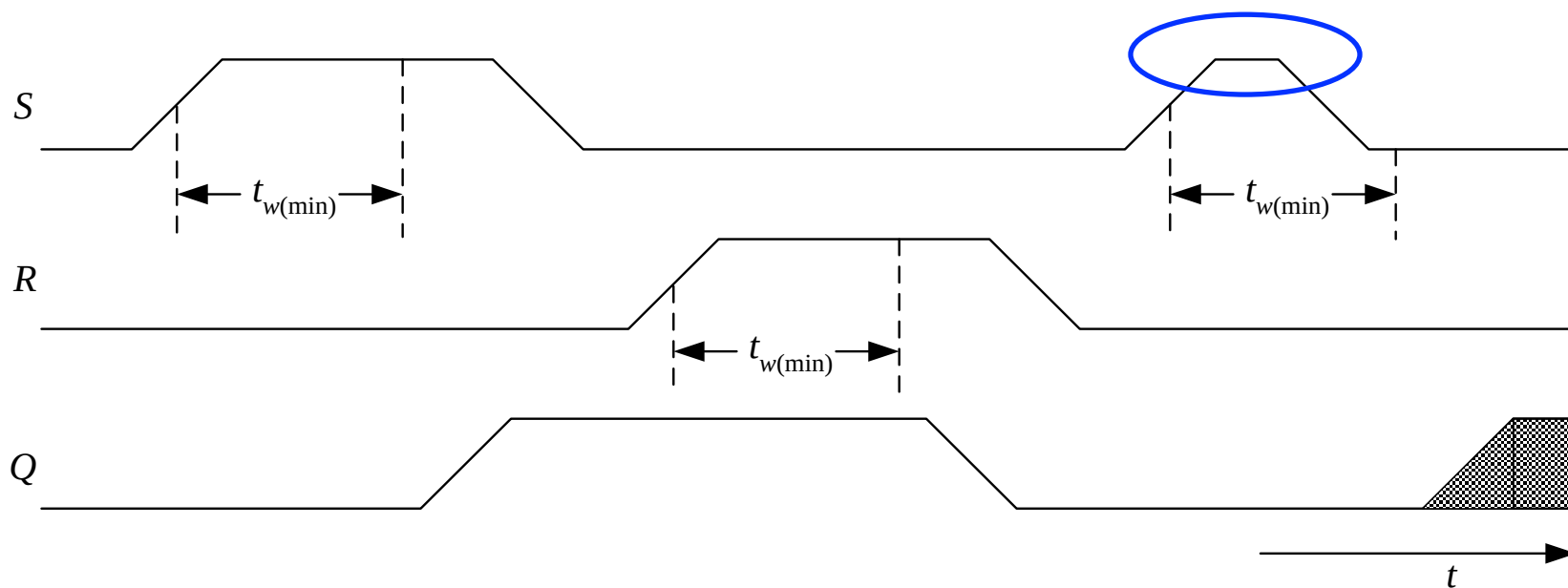
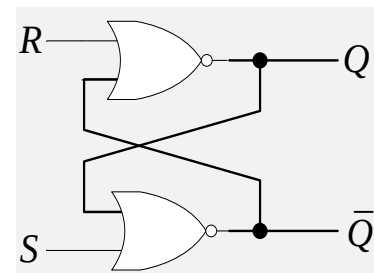
D	C	Q^+	$\bar{Q}^+$
0	1	0	1
1	1	1	0
$\times$	0	Q	$\bar{Q}$

$$Q^+ = D$$

锁存器的时序参数

- 最小脉冲宽度

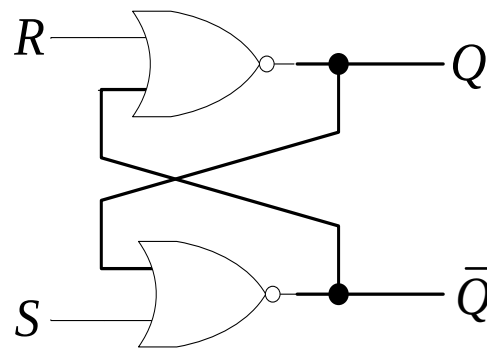
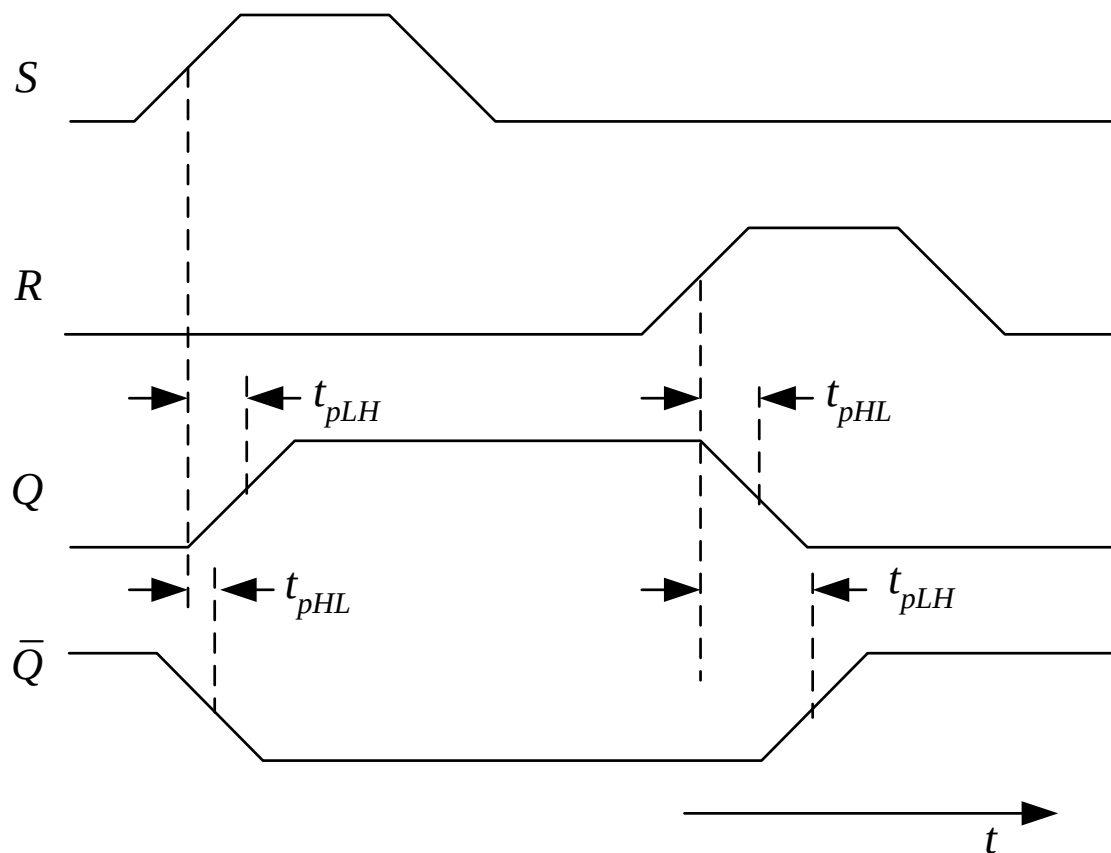
✓ 以根据输入获得所需输出结果



锁存器的时序参数

- 延迟

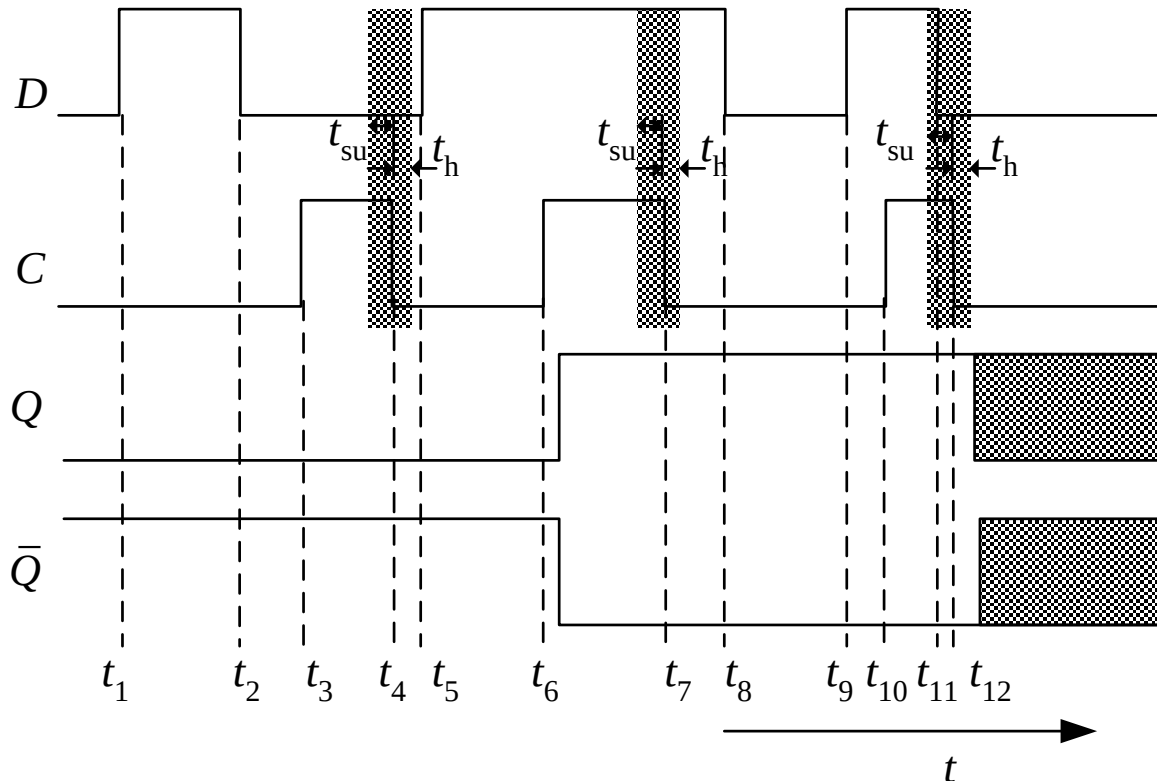
✓ 从输入得到相应输出的时间间隔



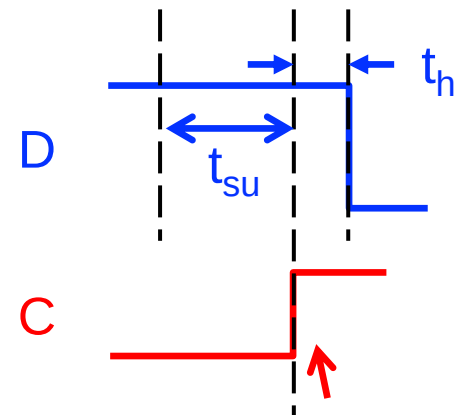
锁存器的时序参数

- 建立时间(Set-up time, t_{su})/保持时间(Hold time, t_h)
 - ✓ 锁存操作开始/结束之前输入信号需要持续的最短时间

Sampling of shaded regions @ neg C

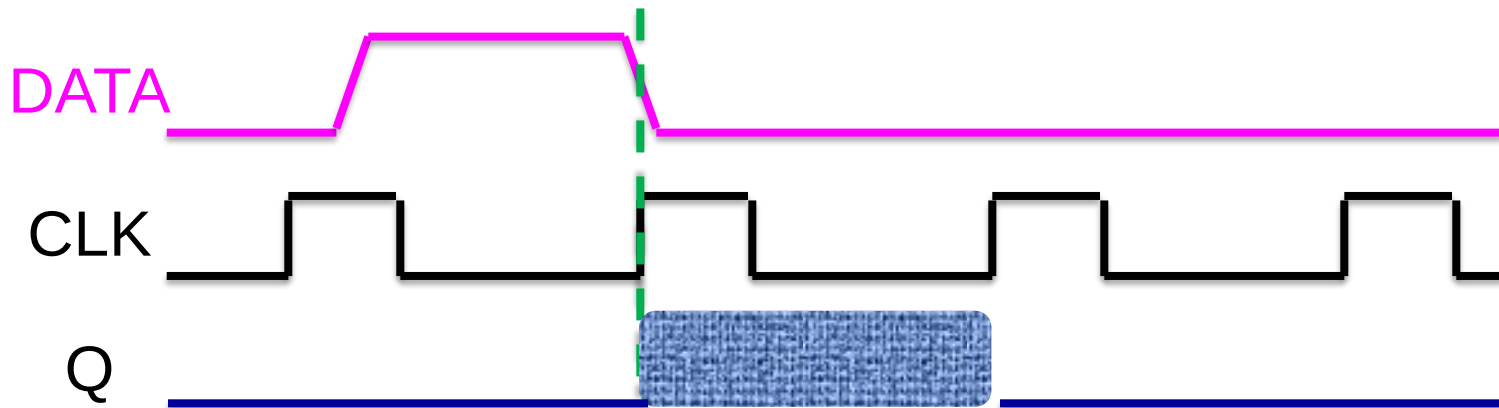


Sampling @ pos C



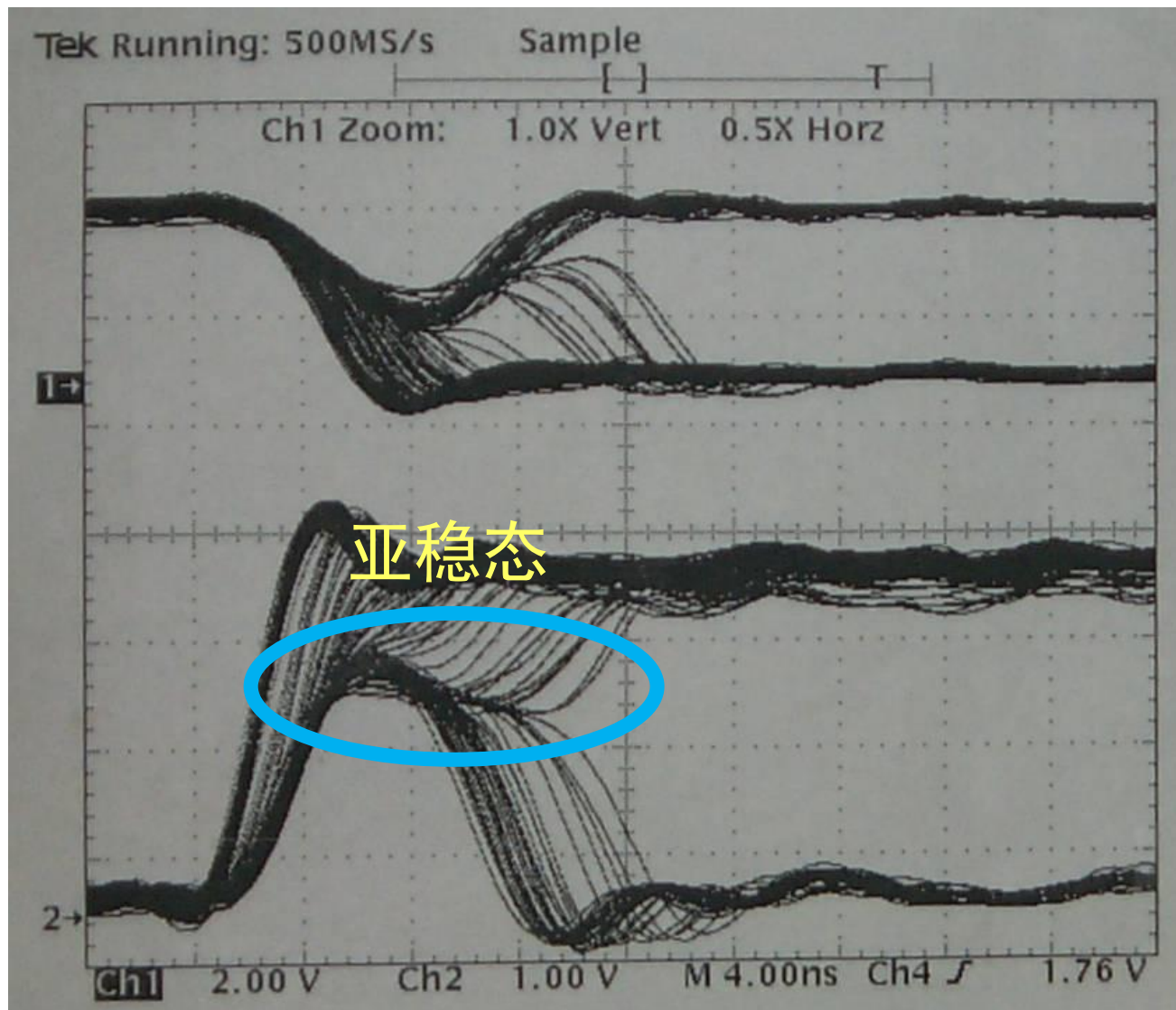
亚稳态

- 如果信号不满足建立时间和保持时间约束
 - ✓ 输出结果亚稳态，不确定：1或0



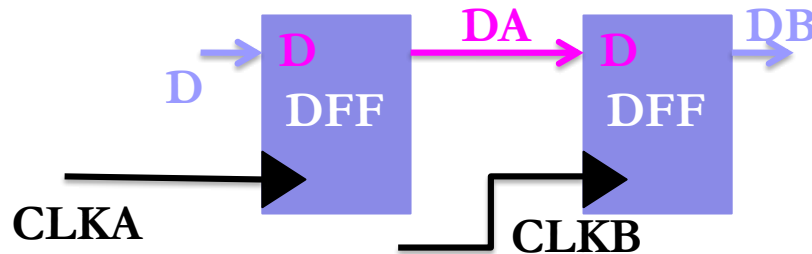
CLK 采样了正在变化的DATA

亚稳态



亚稳态

- 难以完全避免
 - 不同时钟域
 - 时钟偏差
- 通过合理的设计减少/容忍亚稳态
 - 同步器设计
 - 异步FIFO



令 mean time between failure (MTBF) 足够长、满足需求即可。

锁存器总结

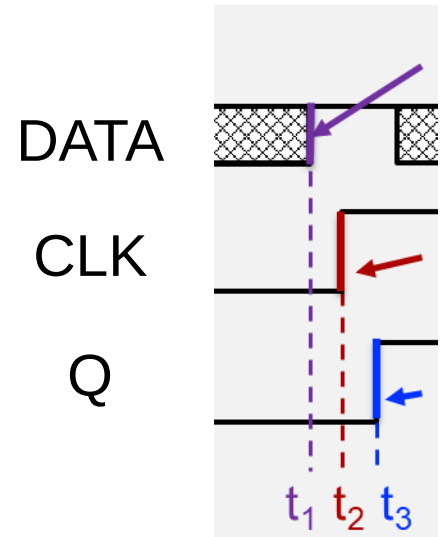
- 目的：将输入信号锁定在Q，可选CLK
- 透明度：监控输入，在一段连续时间内传递给输出
- 时序非常重要
- 作用
 - 计算系统的输入缓存：同步
 - 暂时存储后续操作的输入

本节课目录

- 概念：时钟，状态和FSM
- 时序逻辑电路：锁存器
- 时序逻辑电路：DFF
- 分析，设计，举例

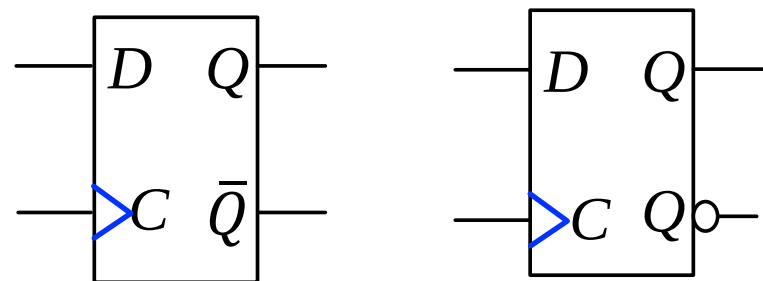
从锁存器到触发器

- 我们需要边沿触发采样！
 - 只在触发沿把输入传递给输出
 - SR锁存器？门控SR锁存器？D锁存器？
- 边沿触发
 - 在时钟上升或者下降沿触发
 - 时序
 - 输入在触发沿“附近”保持不变
 - 输出在一段延时后相应地改变
- 如何实现？
 - 基于Latch？



DFF: 功能和符号

DFF: D flip-flop



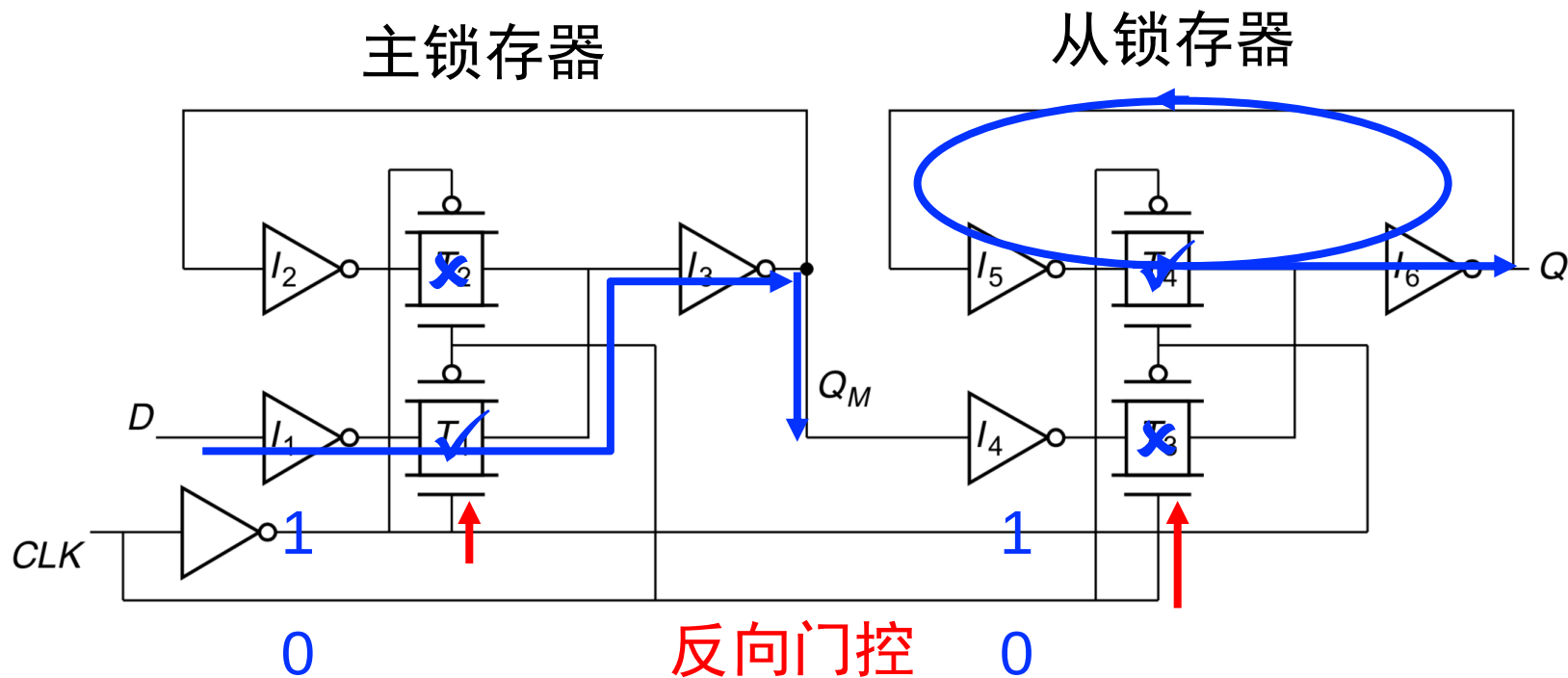
注意与门控D锁存器区别！

D	C	Q^+	$\bar{Q}^+$
0	$\uparrow$	0	1
1	$\uparrow$	1	0
0/1	0/1	Q	$\bar{Q}$

```
module D_ff(clk,D,Q);  
  input clk,D;  
  output reg Q;  
  always@(posedge clk)  
  begin  
    Q <= D;  
  end  
endmodule
```

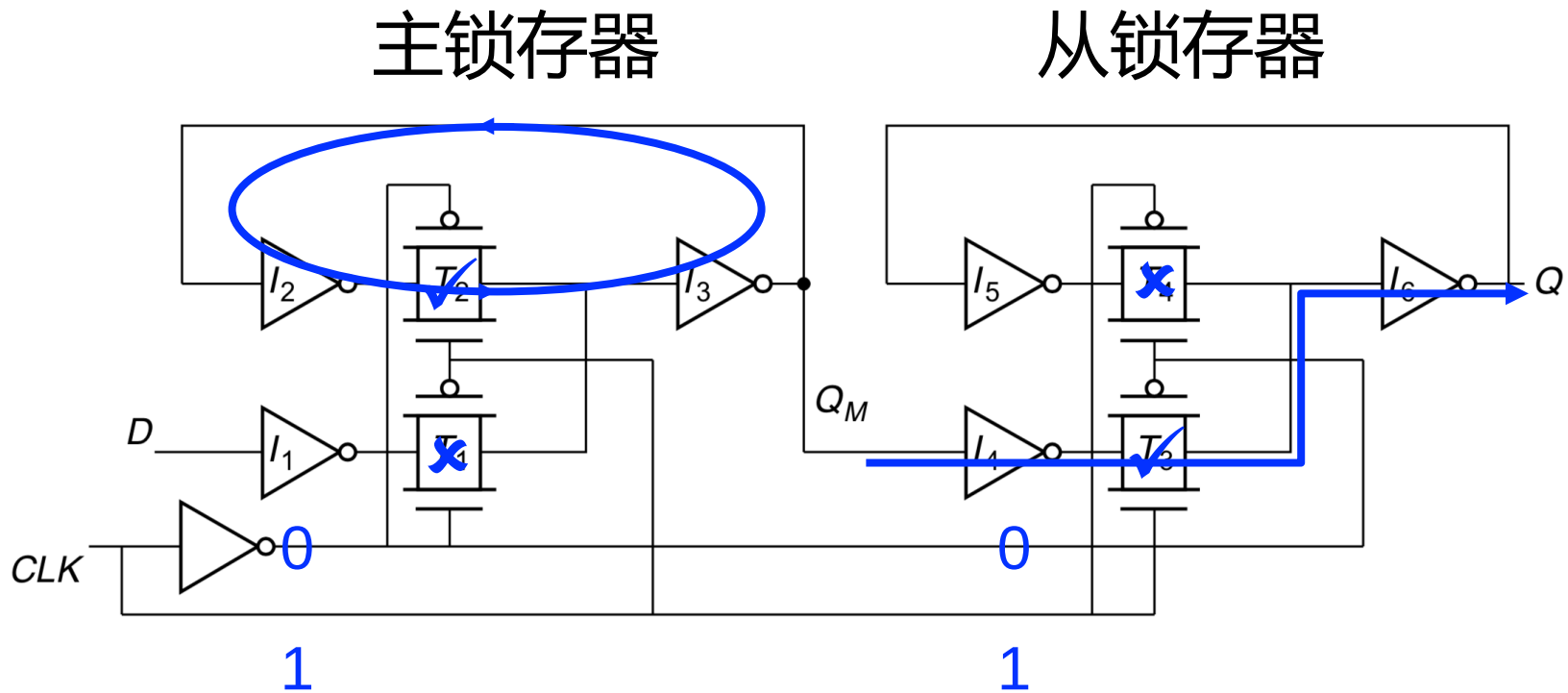
DFF: 常见的一种实现方式

输入数据D，时钟CLK，输出Q



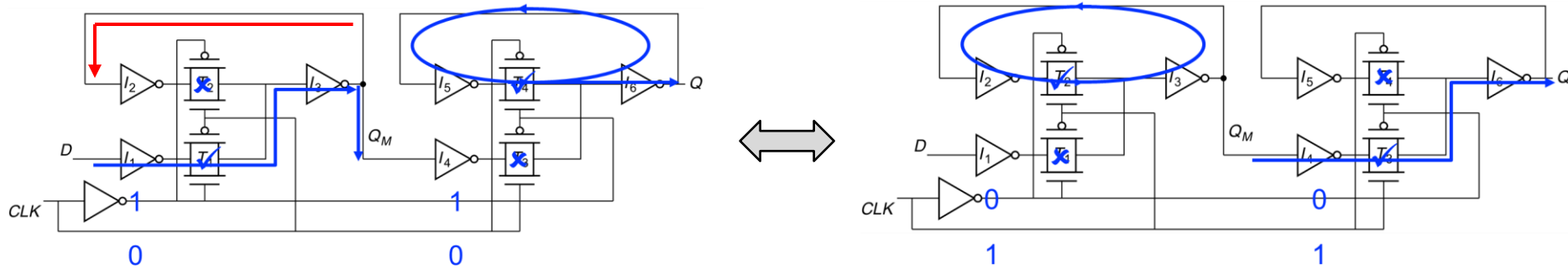
CLK为低: 主锁存器输出 Q_M 随输入 D 变化，从锁存器输出 Q 不变

DFF: 常见的一种实现方式



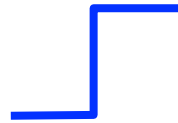
CLK为高: 主锁存器维持在 Q_M , 从锁存器将其传输到 Q

DFF: 常见的一种实现方式



CLK低

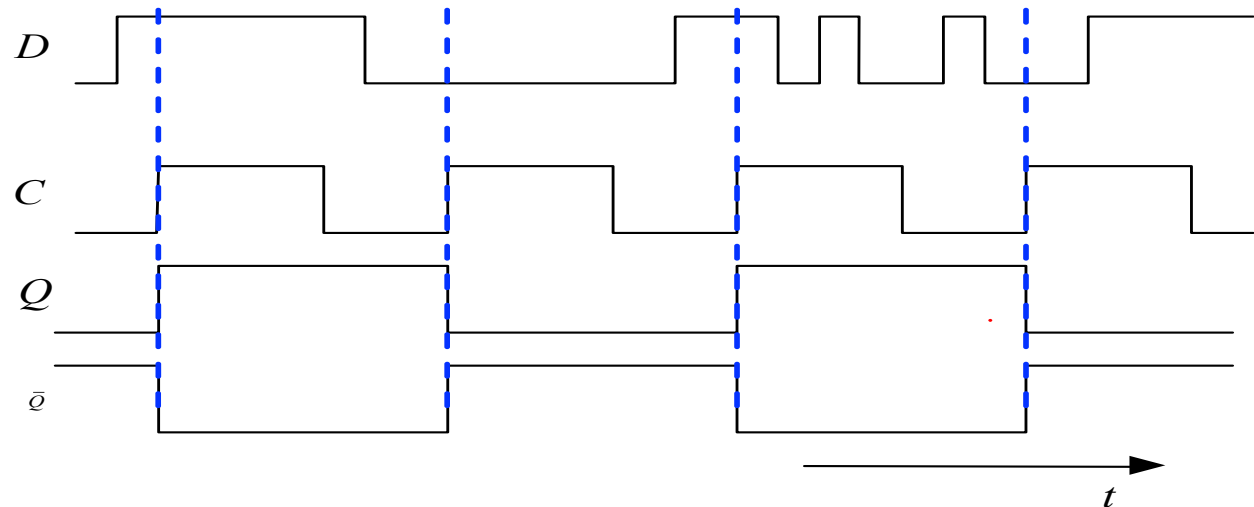
- ✓ Master跟踪D
- ✓ Slave保持原有Q



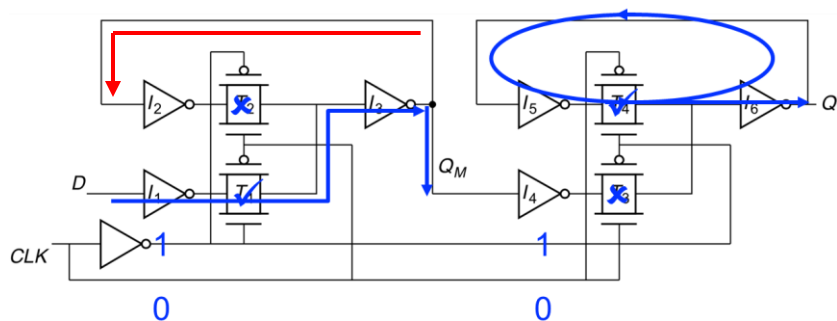
CLK高

- ✓ Master锁定 Q_M
- ✓ Slave传递 Q_M 至Q

理想DFF时序

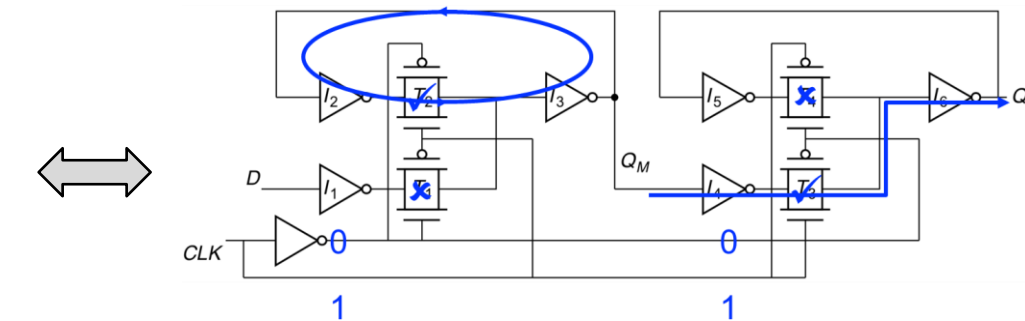


DFF: 时序考虑



CLK低

- ✓ Master跟踪D
- ✓ Slave保持原有Q



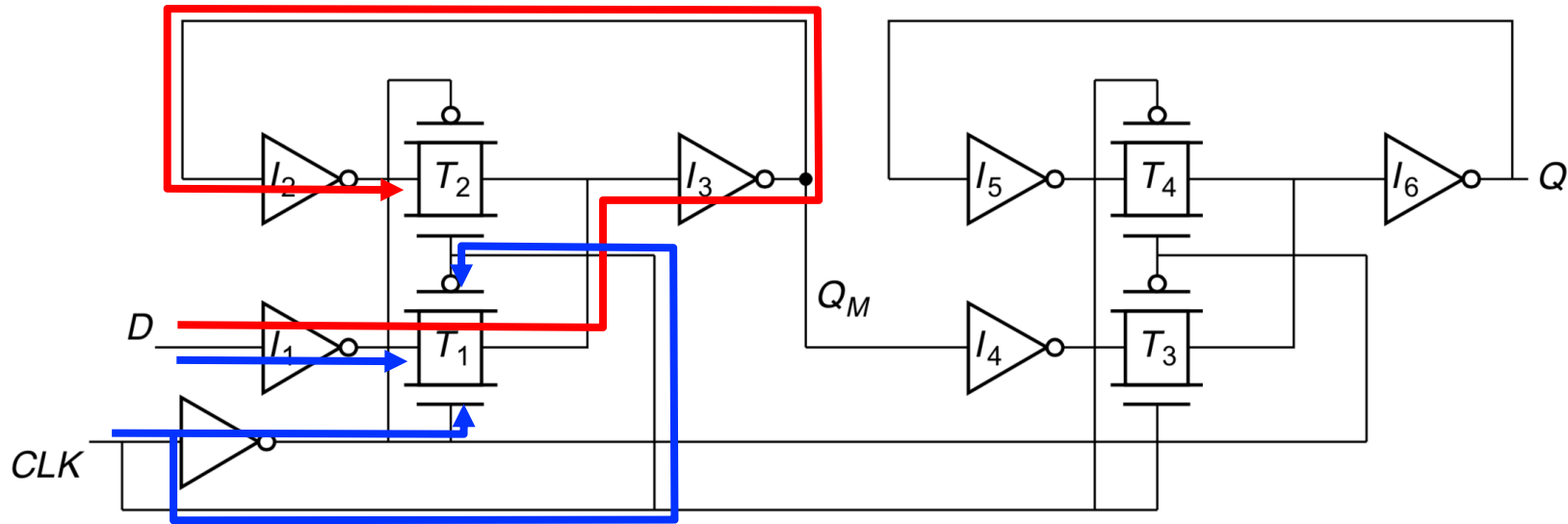
CLK高

- ✓ Master锁定 Q_M
- ✓ Slave传递 Q_M 至Q

如何保证动态过渡过程中功能正确？

- 来自D的输入要在Master锁定时保持稳定
- Master改变 Q_M 前，Slave要完成锁定 (一般原生满足)

DFF: 时序考虑



来自D的输入要在master锁定时保持稳定

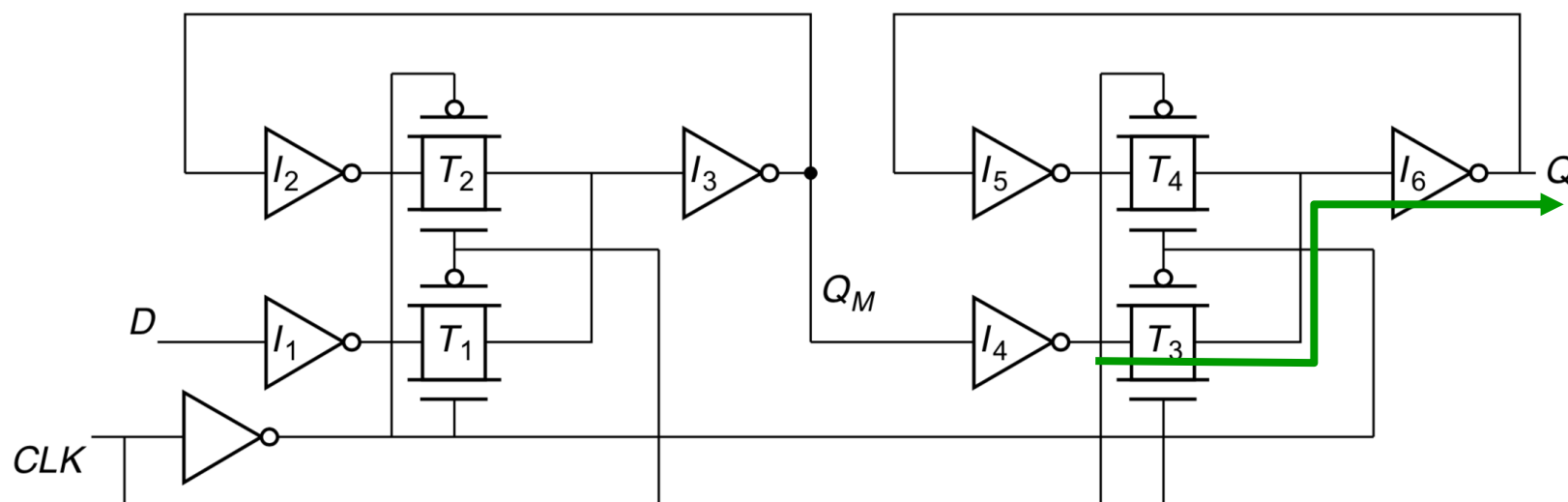
1. 建立时间 (Setup time) before CLK goes high

- ✓ T_2 导通 (CLK上升) 之前, D的输入传递到 T_2 两端, 可以避免竞争

2. 保持时间 (Hold time) after CLK goes high

- ✓ D的输入维持到CLK把 T_1 完全关闭, 可以避免破坏写入的值 (Q_M)
- ✓ D和CLK传播到 T_1 的时间一般相等, 因此通常原生满足 (M-S结构DFF)

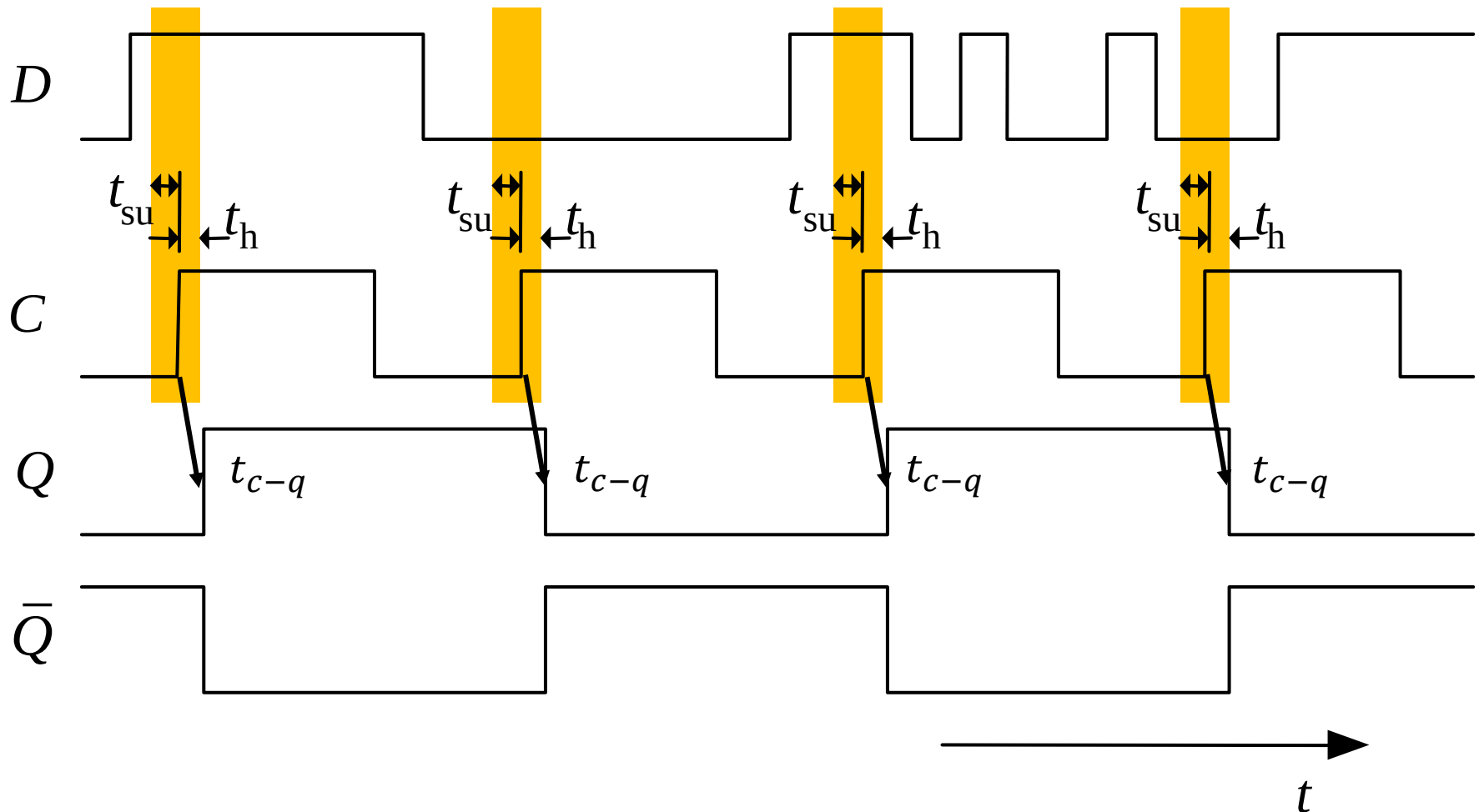
DFF: 时序考虑



3. 输出响应时间 (t_{c-q})

- ✓ 时钟上升后, Q_M 处锁存的数据需要经过 T_3 - I_6 的延时才能到达 Q

DFF: 实际的时序, 存在 t_{su} , t_h , t_{c-q}



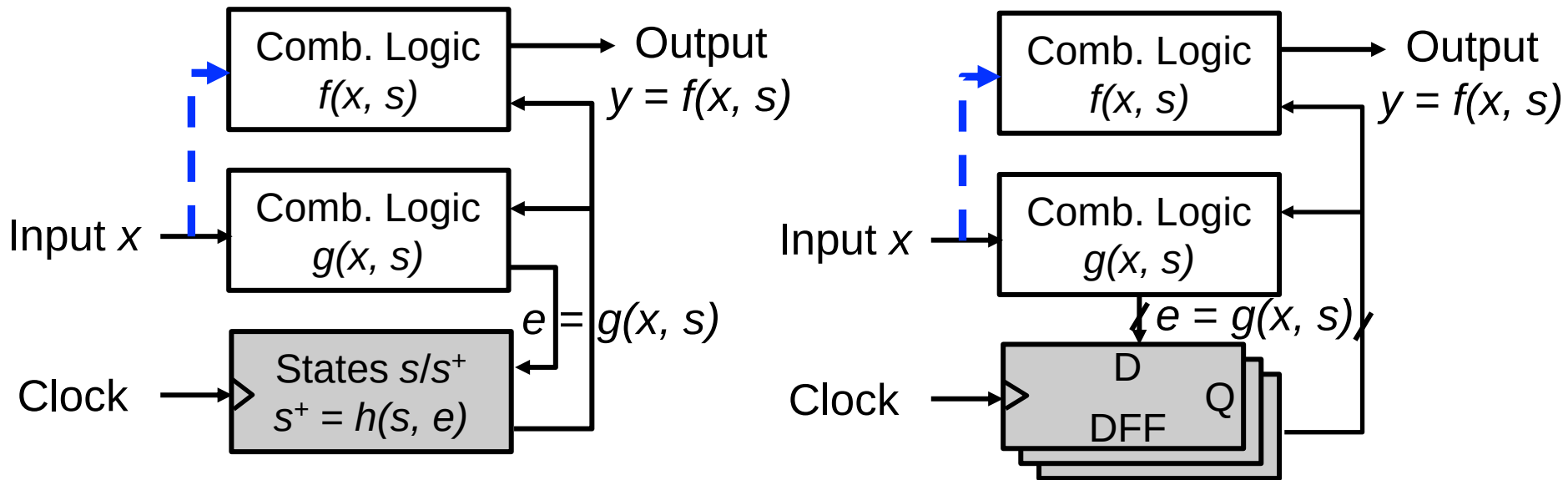
抽象实现中，更新瞬间完成；但是，实际实现中的考虑？

- 每日跑步距离
Input



为什么需要时序？

- 更好组织计算!



✓ D 应该在采样开始之前准备好: $e = f(x, s) \rightarrow D$ 可能因为一个慢的反馈环路花费太多时间!

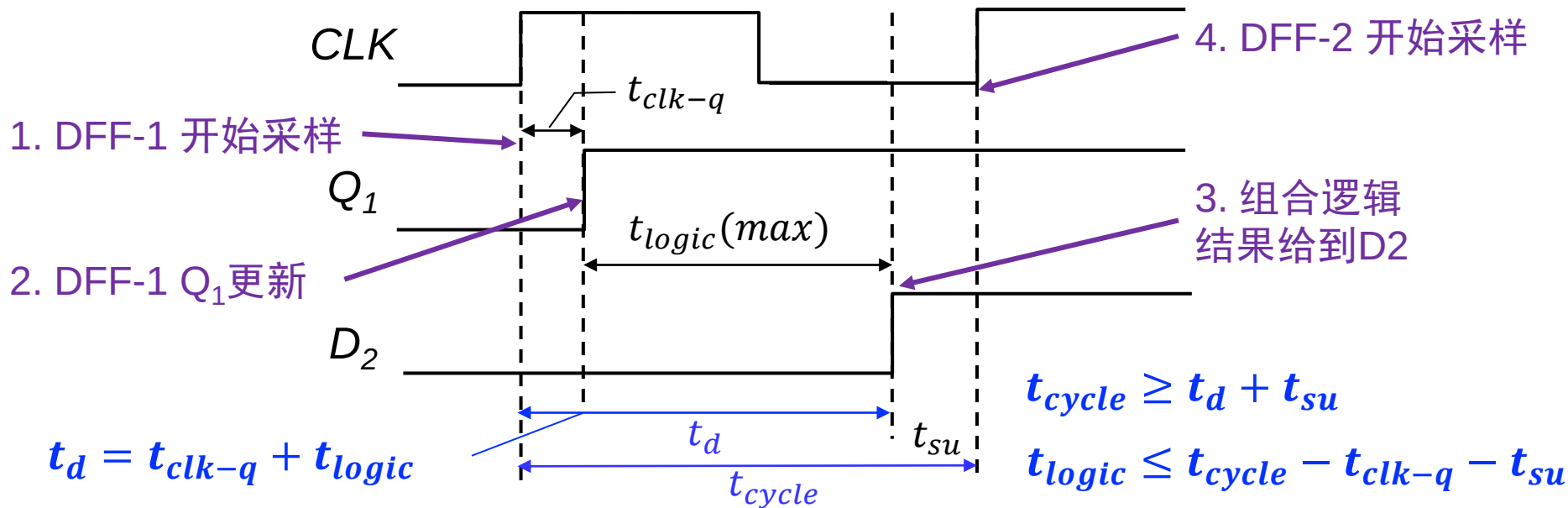
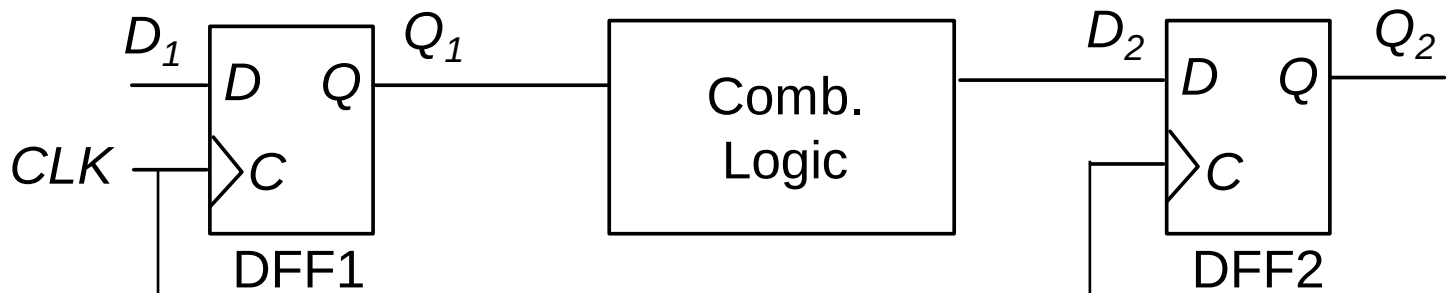
✓ D 应该在采样开始后保持稳定: $e = f(x, s) \rightarrow D$ 可能因为一个快的反馈环路而被破坏!



建立时间约束

• 上升沿触发例子

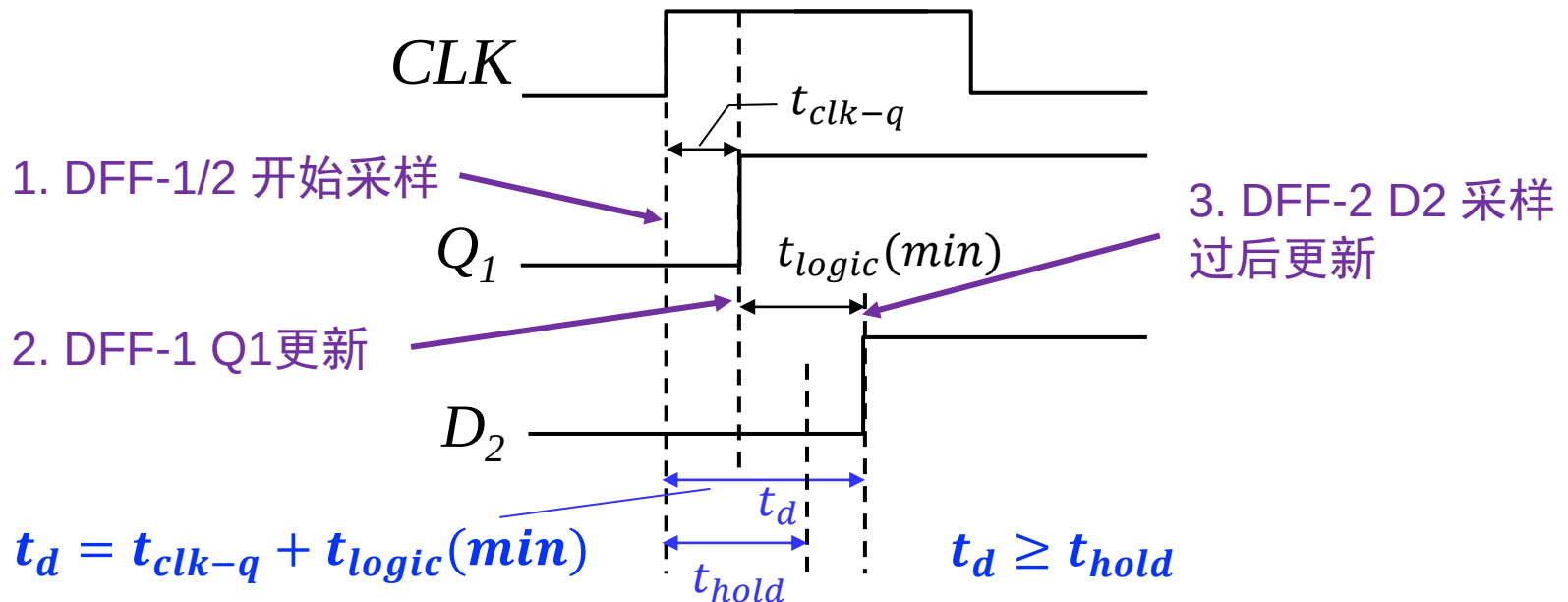
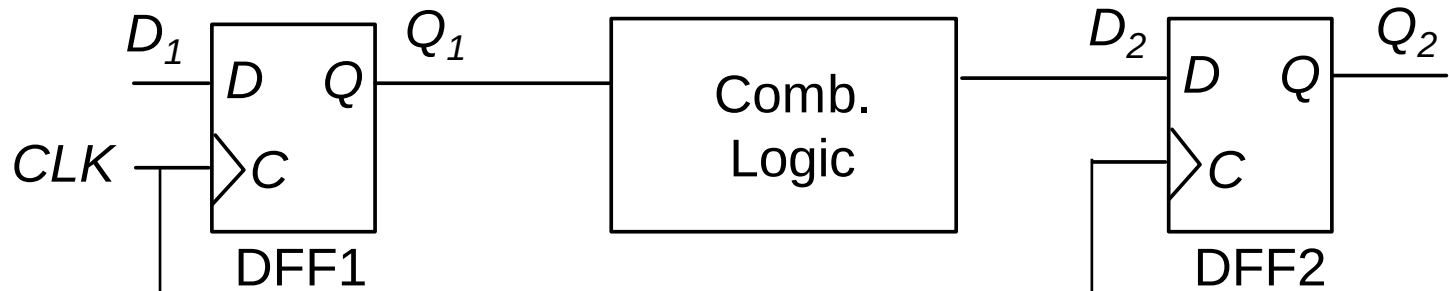
后级DFF的输入信号D必须在触发时钟到来前已经稳定 t_{su} 时间



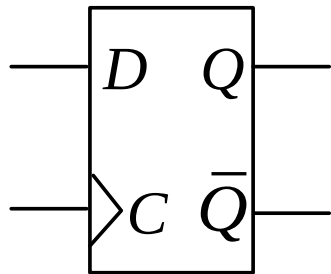
保持时间约束

- 上升沿触发例子

后级DFF的输入信号D必须在触发时钟到来后继续稳定 t_h 时间



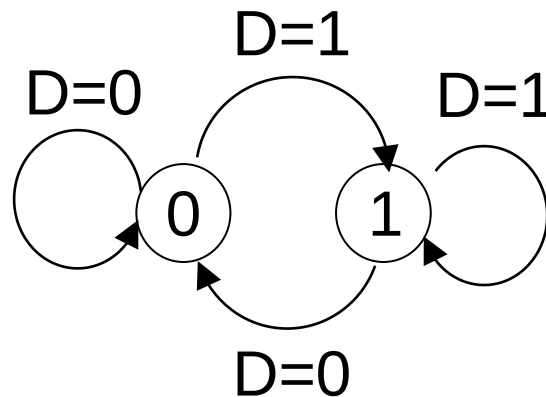
真值表与状态图



D	Q	Q^+
0	0	0
0	1	0
1	0	1
1	1	1

给定输入，计算 $Q \rightarrow Q^+$

Q 和 Q^+ 在不同输入下的对比



特征方程 (Characteristic equations)

Q^+ 与(Q 和输入信号)之间的逻辑表达式

D	0	1
Q		
0	0	1
1	0	1

$$Q^+ = D$$

激励表

根据状态转换寻找激励条件

给定 $Q \rightarrow Q^+$, 寻找输入

$Q \rightarrow Q^+$	D
$0 \rightarrow 0$	0
$0 \rightarrow 1$	1
$1 \rightarrow 0$	0
$1 \rightarrow 1$	1

本讲主要内容

- 概念：时钟，状态和FSM
- 时序逻辑电路：锁存器
- 时序逻辑电路：DFF
- 分析，设计，举例

作业

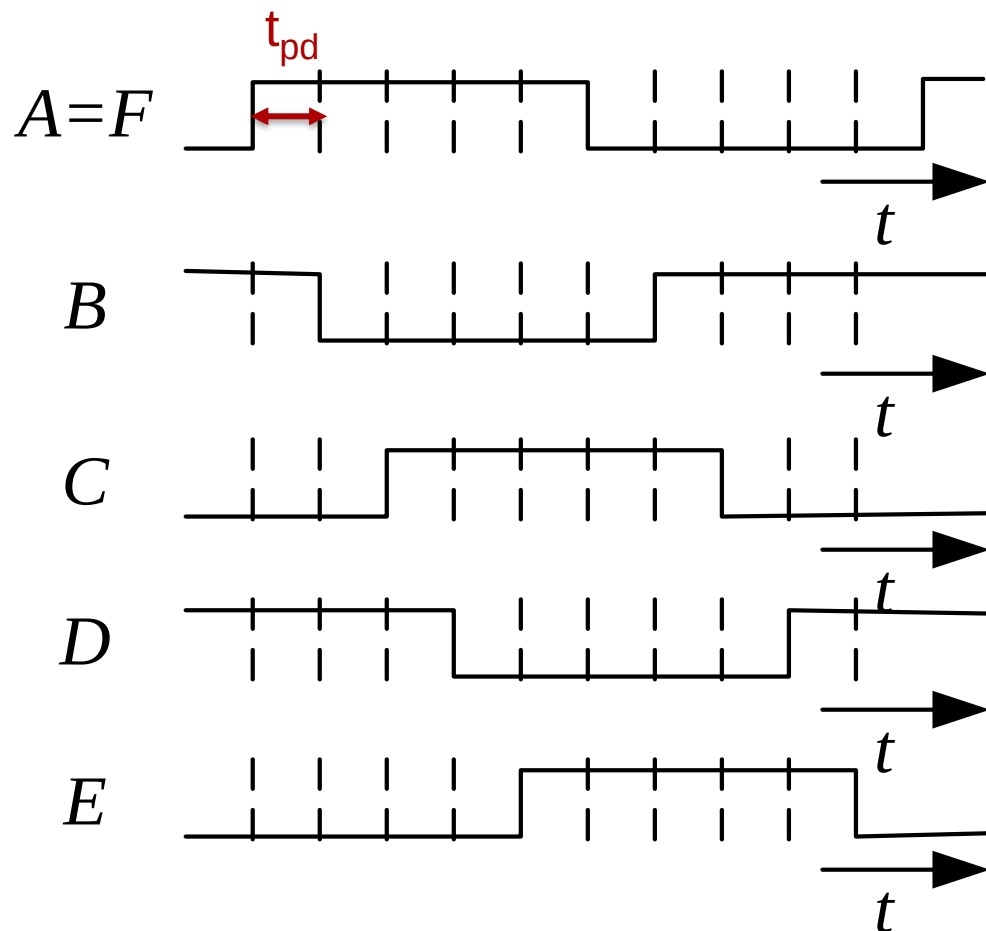
- 要求
 - 线上交电子版
 - 不要抄袭，迟交一周内扣50%，答案公布后再交不给分
- 本次作业
 - 网络学堂作业区

谢谢大家！

- 课后阅读1: 其他类型的触发器及相互转换
- 课后阅读2: 不同触发器原理的差异
- 课后阅读不考查

环形振荡器相位

- 一组相位不同的时钟信号



$$y = f\left(\frac{2\pi}{T}t + \varphi\right)$$

周期 (T)? $10t_{pd}$

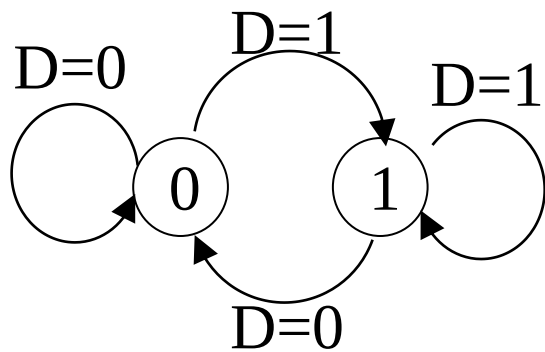
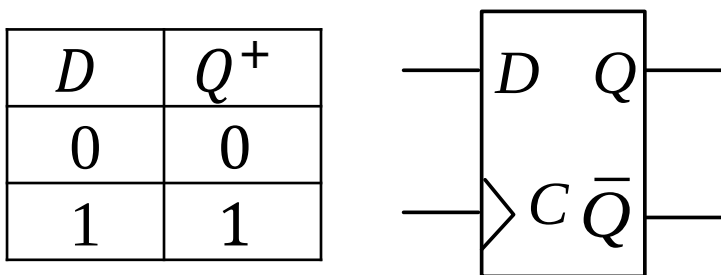
占空比 (f)? 50% , 方波

相位 (φ)?

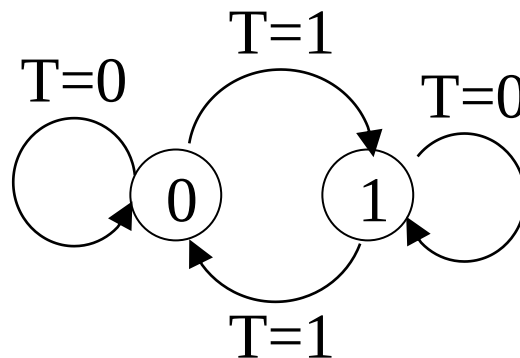
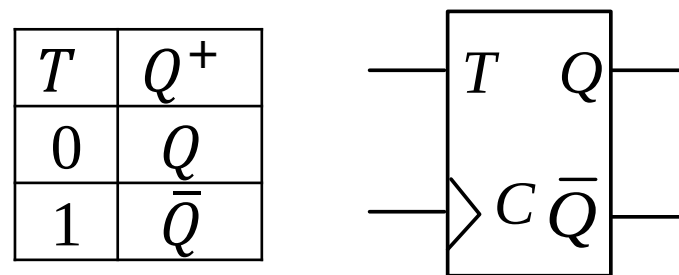
四种触发器：DFF和T-FF

真值表和状态图

D flip-flop:



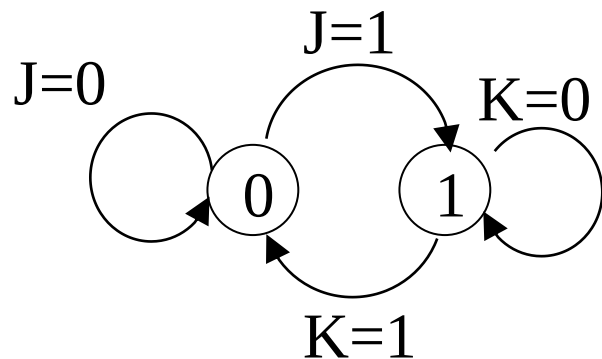
T flip-flop:



四种触发器：JK-FF和RS-FF

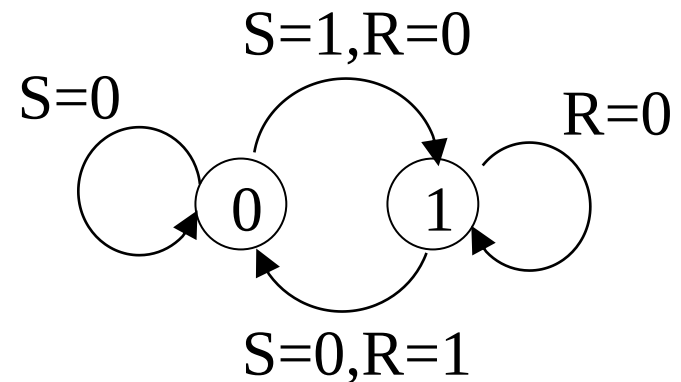
JK flip-flop:

J	K	Q^+
0	0	Q
0	1	0
1	0	1
1	1	$\bar{Q}$

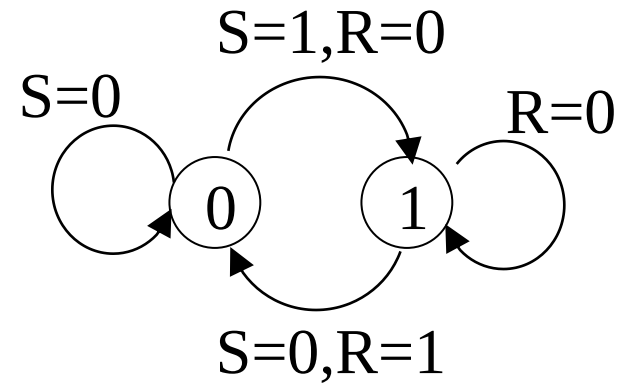
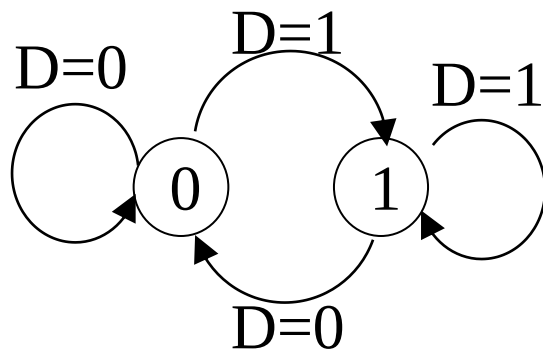
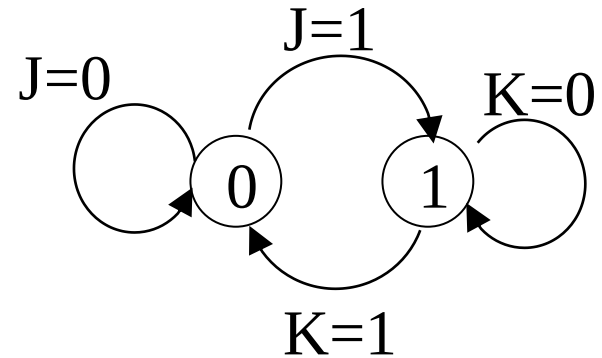
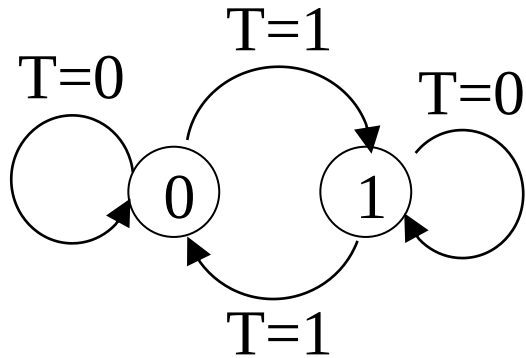


RS flip-flop:

S	R	Q^+
0	0	Q
0	1	0
1	0	1
1	1	-



状态图 (State diagram)



真值表

T	Q	Q^+		D	Q	Q^+
0	0	0		0	0	0
0	1	1		0	1	0
1	0	1		1	0	1
1	1	0		1	1	1

给定输入，计算 $Q \rightarrow Q^+$

Q 和 Q^+ 在不同输入
下的对比

S	R	Q	Q^+		J	K	Q	Q^+
0	0	0	0		0	0	0	0
0	0	1	1		0	0	1	1
0	1	0	0		0	1	0	0
0	1	1	0		0	1	1	0
1	0	0	1		1	0	0	1
1	0	1	1		1	0	1	1
1	1	0	-		1	1	0	1
1	1	1	-		1	1	1	0

特征方程 (Characteristic equations)

Q^+ 与(Q 和输入信号)之间的逻辑表达式

D	0	1
Q		
0	0	1
1	0	1

$$Q^+ = D$$

T	0	1
Q		
0	0	1
1	1	0

$$\begin{aligned} Q^+ &= T\bar{Q} + \bar{T}Q \\ &= T \oplus Q \end{aligned}$$

特征方程

RS	00	01	11	10
Q				
0	0	1	-	0
1	1	1	-	0

$$Q^+ = S + \bar{R}Q$$

$$SR = 0$$

JK	00	01	11	10
Q				
0	0	0	1	1
1	1	0	0	1

$$Q^+ = J\bar{Q} + \bar{K}Q$$

激励表

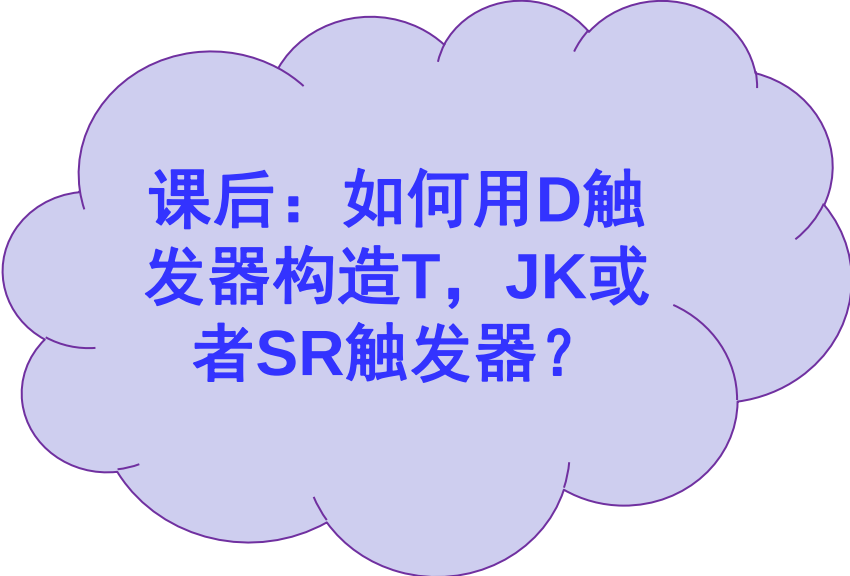
根据状态转换寻找激励条件

给定 $Q \rightarrow Q^+$, 寻找输入

$Q \rightarrow Q^+$	D	T	J	K	S	R
$0 \rightarrow 0$	0	0	0	-	0	-
$0 \rightarrow 1$	1	1	1	-	1	0
$1 \rightarrow 0$	0	1	-	1	0	1
$1 \rightarrow 1$	1	0	-	0	-	0

用DFF构造其他触发器

- 根据其他触发器的特征方程，用DFF设计这些触发器
 - $D \rightarrow T$
 - $D \rightarrow JK$
 - $D \rightarrow RS$

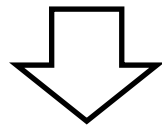


课后：如何用D触发器构造T，JK或者SR触发器？

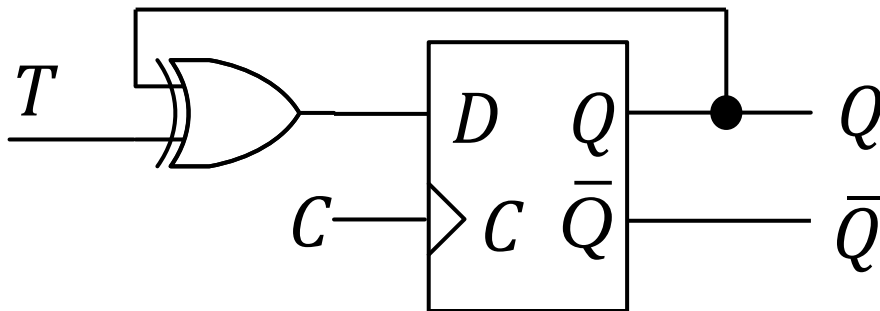
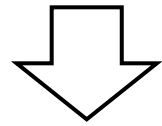
用DFF构造其他触发器

用触发器： $Q^+ = D$

构造T触发器： $Q^+ = \bar{T}Q + \bar{Q}T = T \oplus Q$



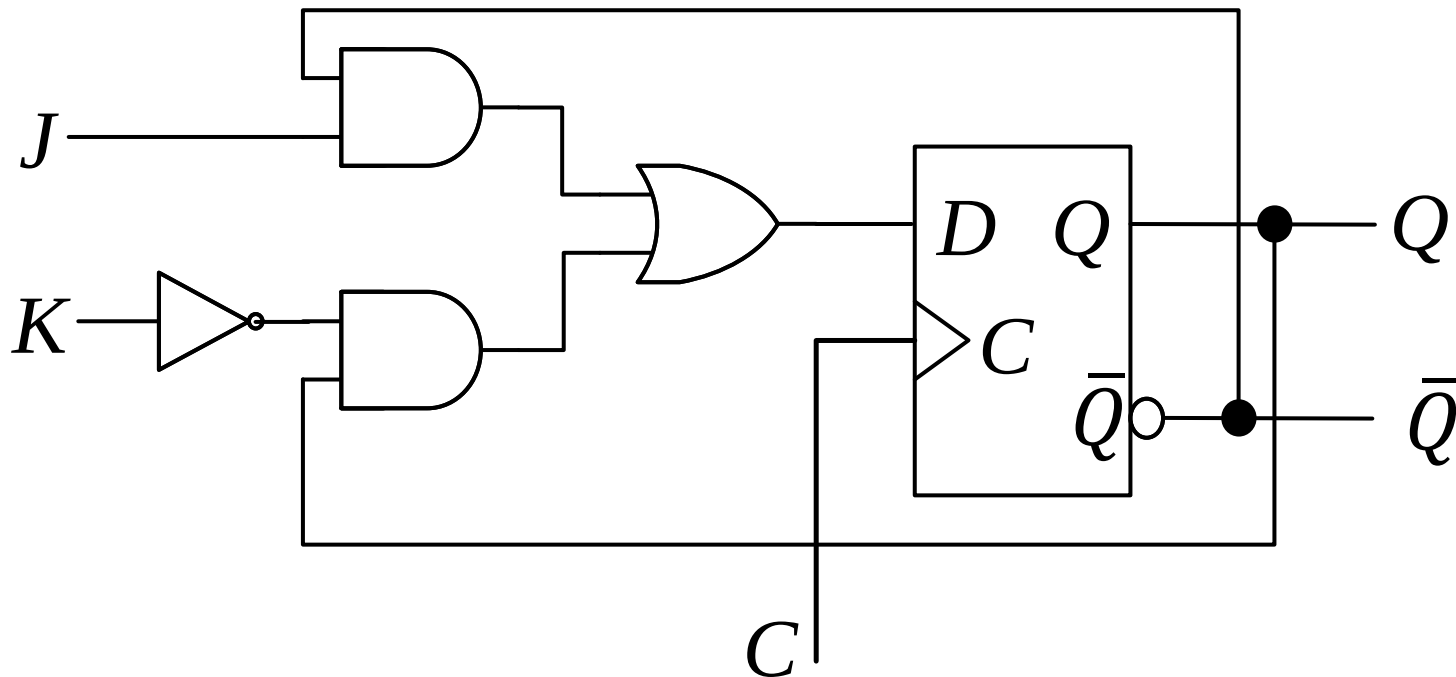
In DFF, let $D = T \oplus Q$



T	C	Q^+	$\bar{Q}^+$
0	$\uparrow$	Q	$\bar{Q}$
1	$\uparrow$	$\bar{Q}$	Q
$\times$	0	Q	$\bar{Q}$
$\times$	1	Q	$\bar{Q}$

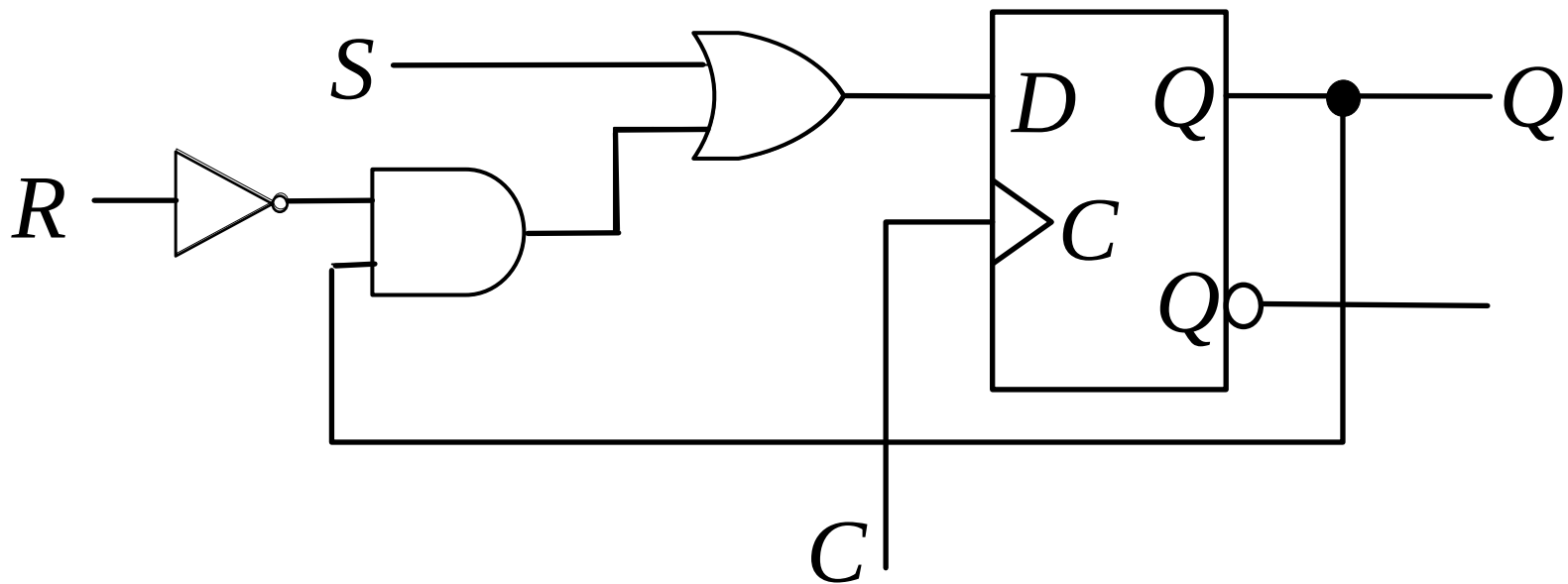
用DFF构造其他触发器

$$\begin{aligned} Q^+ &= D \\ Q^+ &= J\bar{Q} + \bar{K}Q \end{aligned} \quad \Rightarrow \quad D = J\bar{Q} + \bar{K}Q$$



用DFF构造其他触发器

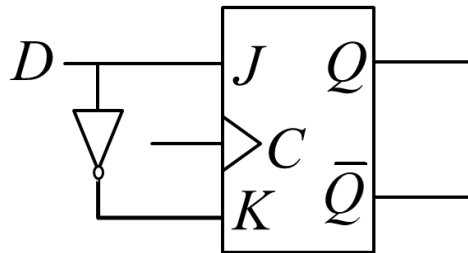
$$\begin{array}{l} Q^+ = S + \bar{R}Q \\ Q^+ = D \end{array} \quad \Rightarrow \quad D = S + \bar{R}Q$$



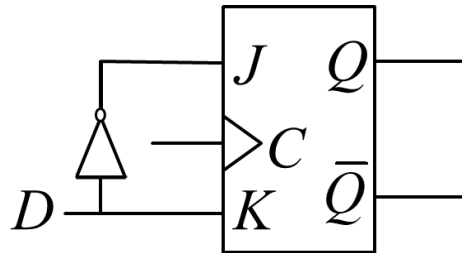
答案解析

请问哪个用JK触发器实现了D触发器？

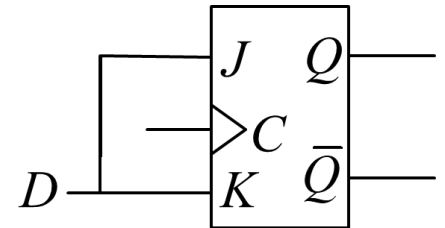
- A.



B.



C.



• 解答：A

- 当 $J=D$, $K=\bar{D}$, JK触发器的输出是D，和D触发器一样。

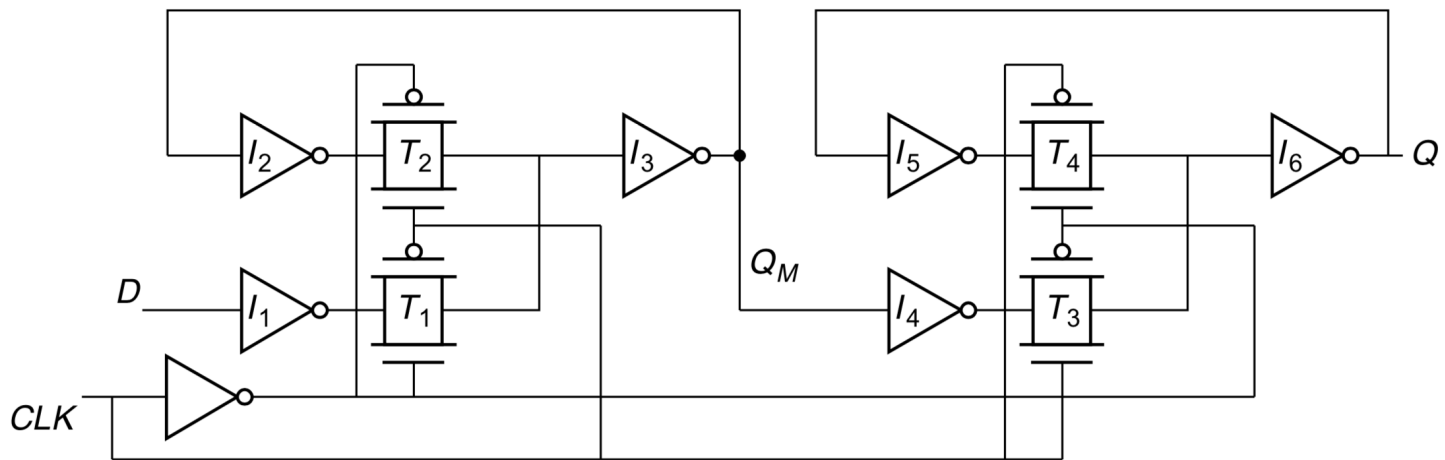
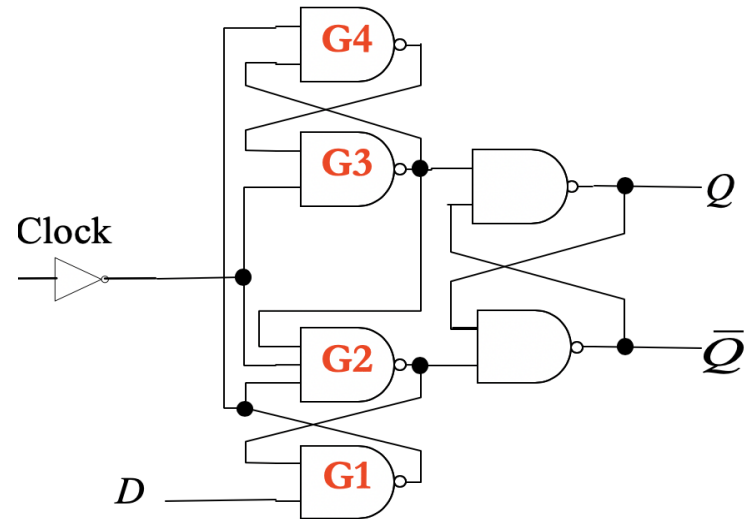
$$Q^+ = J\bar{Q} + \bar{K}Q = DQ' + \bar{\bar{D}}Q = D$$

D	C	Q^+	$\bar{Q}^+$
0	$\uparrow$	0	1
1	$\uparrow$	1	0
$\times$	0	Q	$\bar{Q}$
$\times$	1	Q	$\bar{Q}$

D触发器

不同的DFF实现

```
module D_ff(clk,D,Q);  
  input clk,D;  
  output reg Q;  
  always@(posedge clk)  
  begin  
    Q <= D;  
  end  
endmodule
```

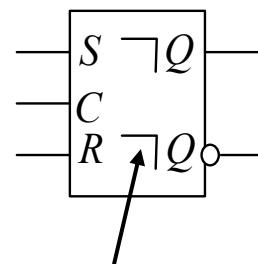
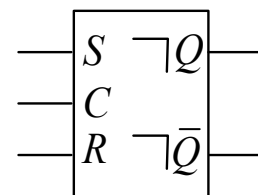
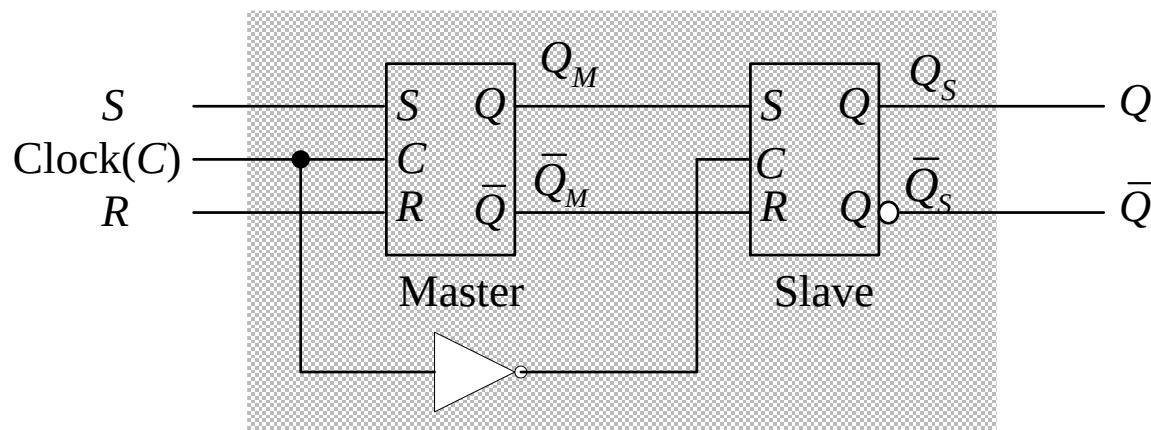


基于传输门逻辑的锁存器对

例1：主从SR触发器

- 图示锁存器的整体行为是输出在时钟下降沿与输入相同，其他时间保持不变
- 概述：所有使用的锁存器只能在输入使能信号为高时才可以改变状态

$$Q^+ = S + \bar{R}Q$$



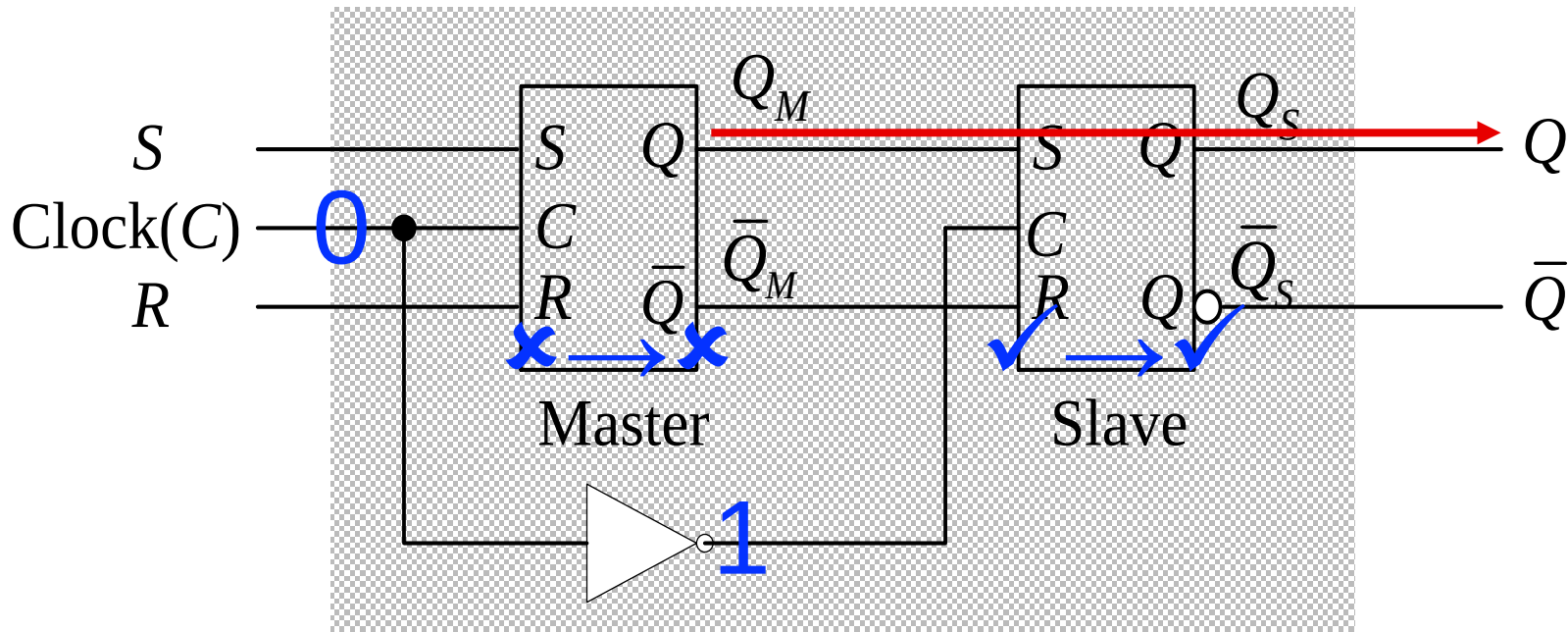
输出延时指示

主从SR触发器的工作流程(下降沿触发)

- 概述：所有使用的锁存器只能在输入使能信号为高时才可以改变状态
- 当Clock为低(0)，主锁存器锁定并保持 Q_M ，从锁存器跟踪($Q_S^+ = Q_M$)，触发器整体表现为锁定 $Q^+ = Q_{M,0}$

主锁存器锁定： $Q^+_M = Q_{M,0}$

从锁存器跟踪： $Q^+ = Q^+_S = Q_M$

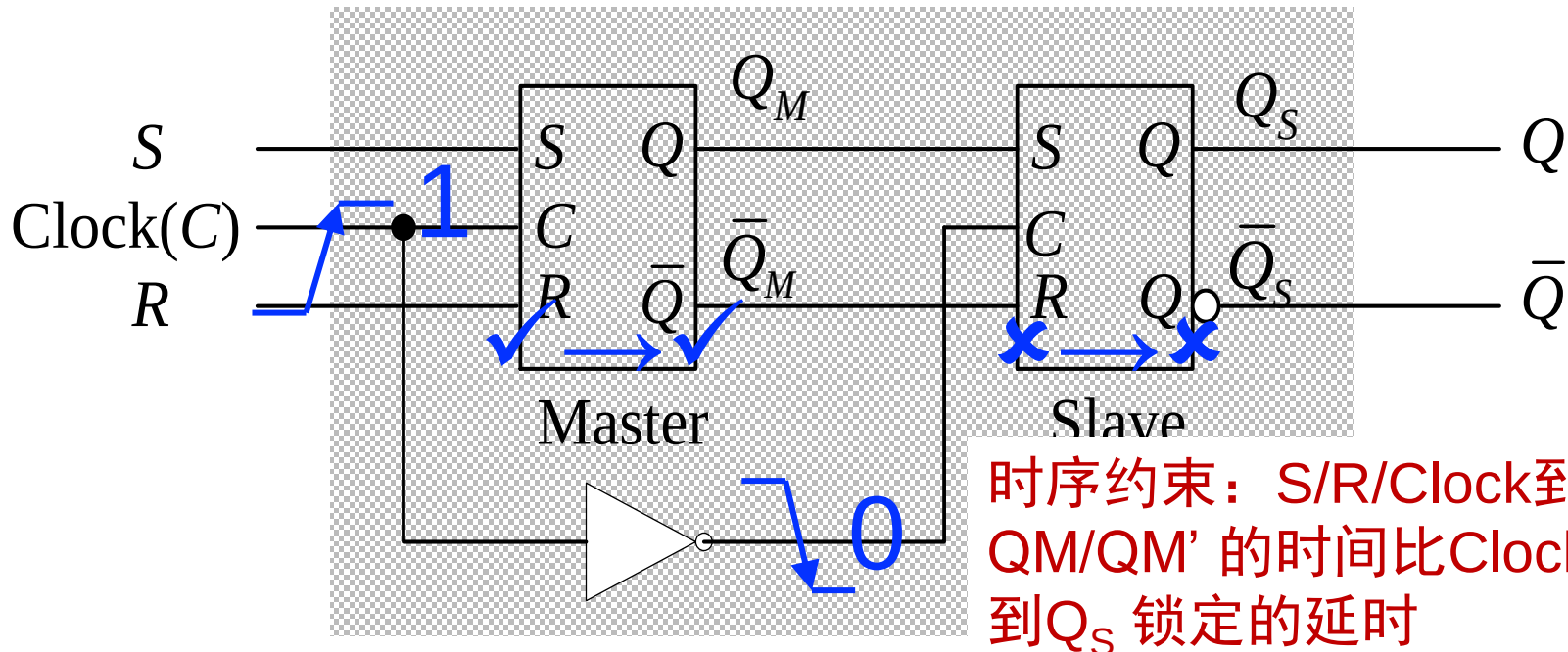


主从SR触发器的工作流程(下降沿触发)

- 概述：所有使用的锁存器只能在输入使能信号为高时才可以改变状态
- 当Clock为低(0)，主锁存器锁定并保持 Q_M ，从锁存器跟踪($Q_S^+ = Q_M$)，触发器整体表现为锁定 $Q^+ = Q_{M,0}$
- 时钟从0变为1，从锁存器锁定，首先保存上一时刻 Q_M 的值；主锁存器变为跟踪状态 (并且一直更新 Q_M^+ 的状态)

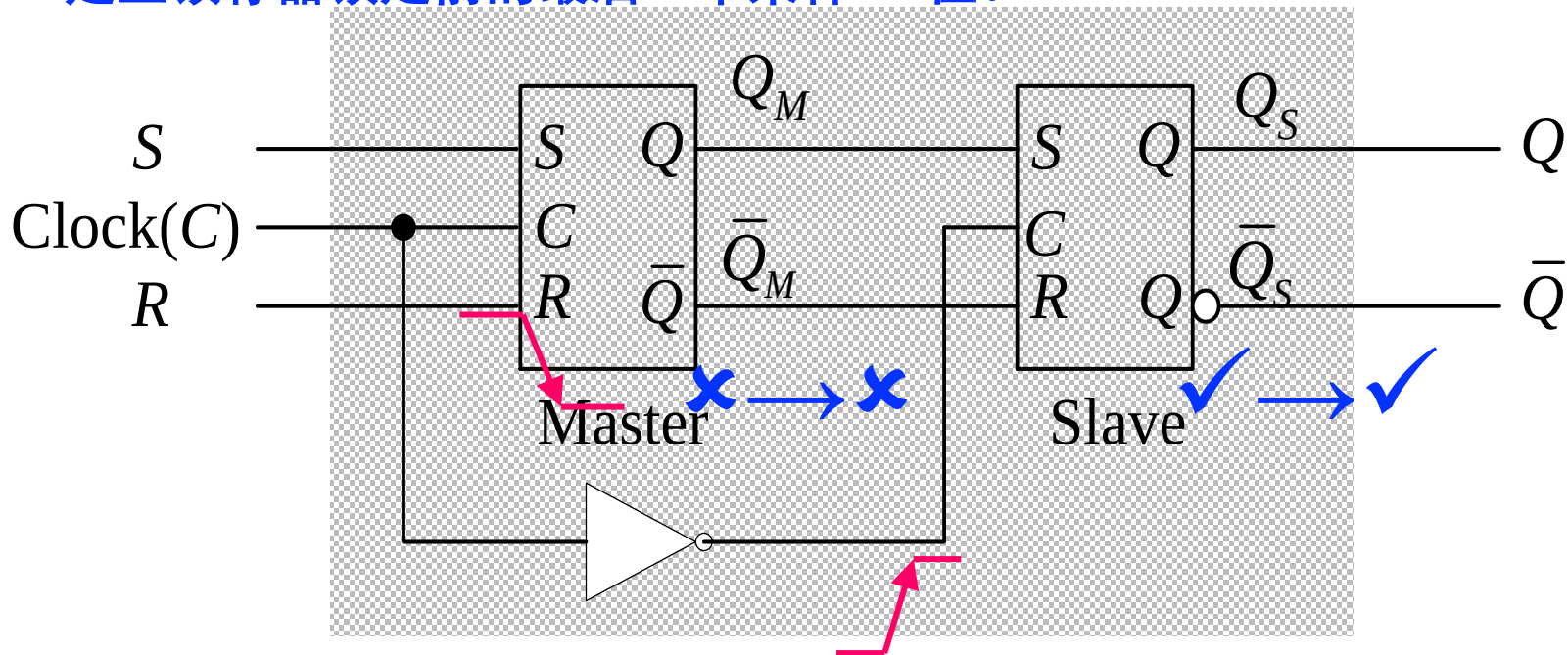
主锁存器跟踪： $Q_M^+ = S + R'Q_M$

从锁存器锁定： $Q_S^+ = Q_M$



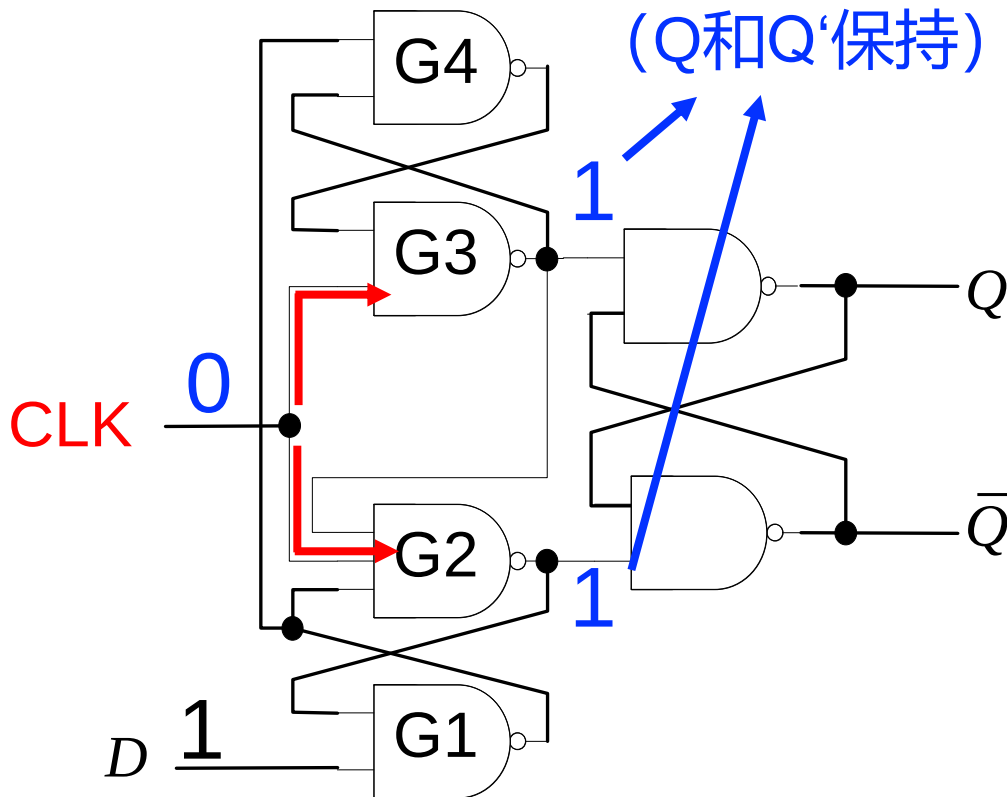
主从SR触发器的工作流程(下降沿触发)

- 概述：所有使用的锁存器只能在输入使能信号为高时才可以改变状态
- 当Clock为低(0)，主锁存器锁定并保持 Q_M ，从锁存器跟踪($Q_S^+ = Q_M$)，触发器整体表现为锁定 $Q^+ = Q_{M,0}$
- 时钟从0变为1，从锁存器锁定，首先保存上一时刻 Q_M 的值；主锁存器变为跟踪状态(并且一直更新 Q_M^+ 的状态)
- **Clock从1到0，主锁存器变回锁定状态，从锁存器变回跟踪状态， Q_S 输出是主锁存器锁定前的最后一个采样SR值。**



边沿触发DFF

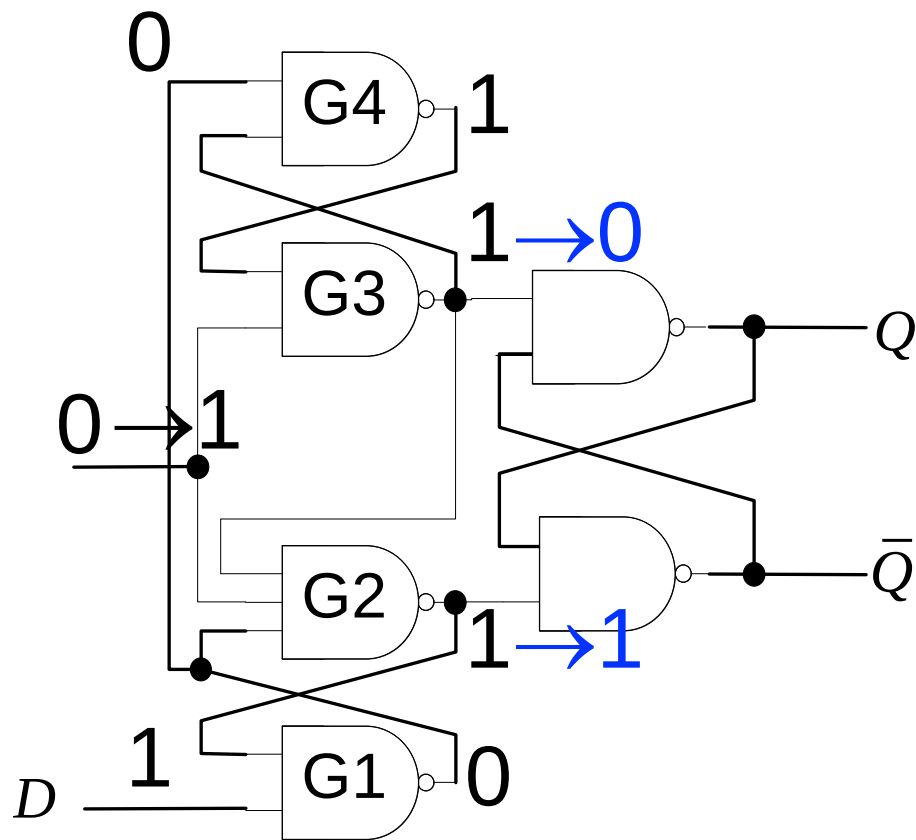
初始状态：假定
这两个节点为1
(Q和Q'保持)



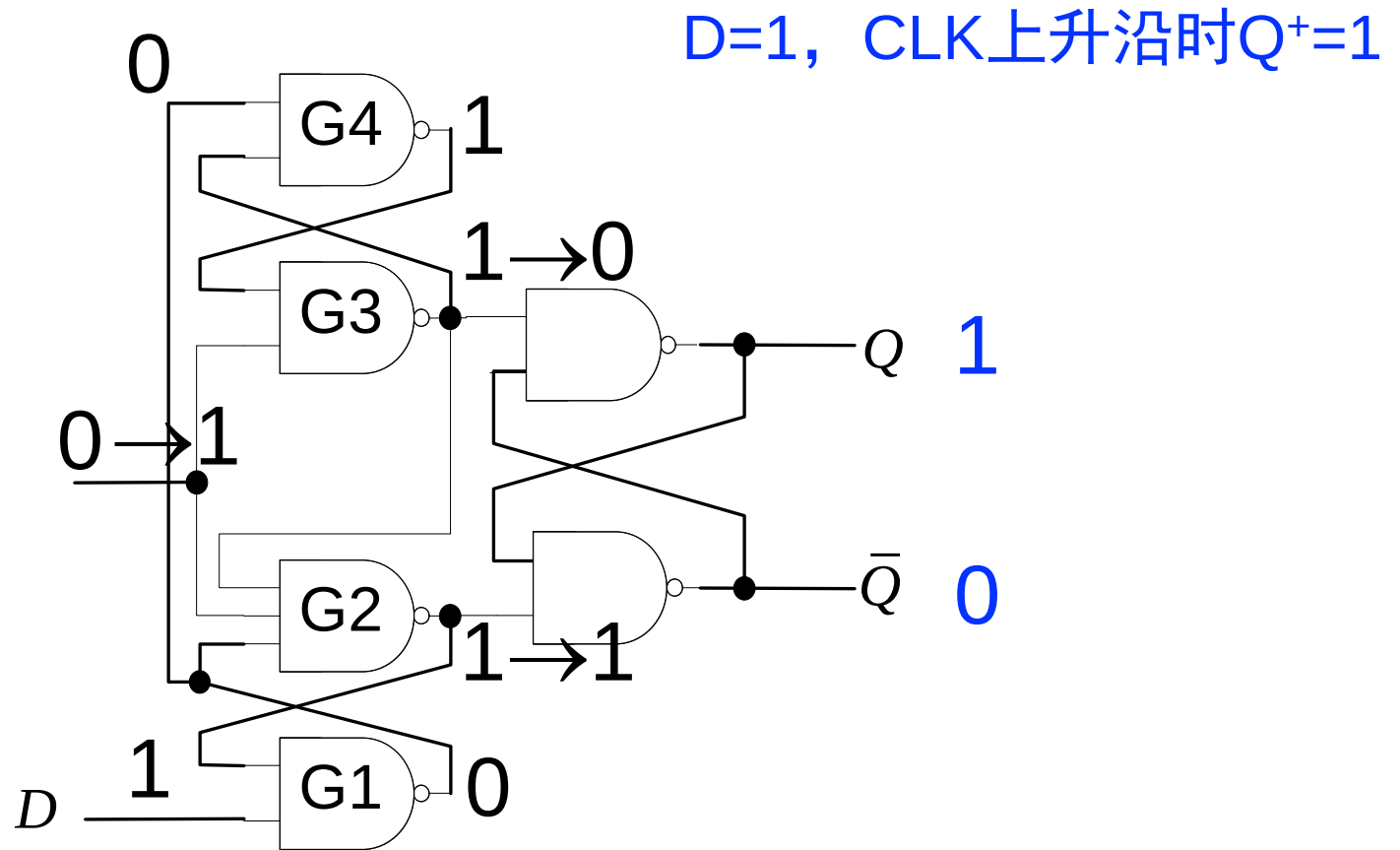
```
module D_flip_flop(clk,D,Q,Q_bar);  
    input clk,D;  
    output Q,Q_bar;  
    wire g1,g2,g3,g4;  
    nand n1(g1,D,g2);  
    nand n2(g2,g1,clk,g3);  
    nand n3(g3,clk,g4);  
    nand n4(g4,g1,g3);  
    nand n5(Q,g3,Q_bar);  
    nand n6(Q_bar,g2,Q);  
endmodule
```

```
module D_ff(clk,D,Q);  
    input clk,D;  
    output reg Q;  
    always@(posedge clk)  
    begin  
        Q <= D;  
    end  
endmodule
```

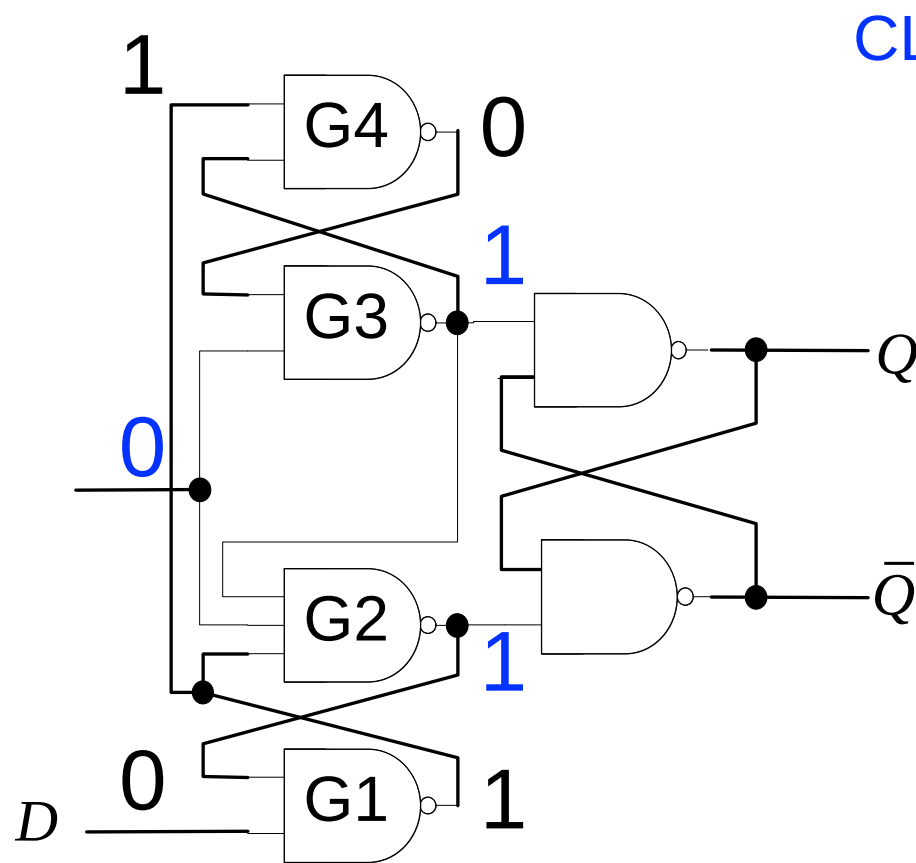
边沿触发DFF



边沿触发DFF

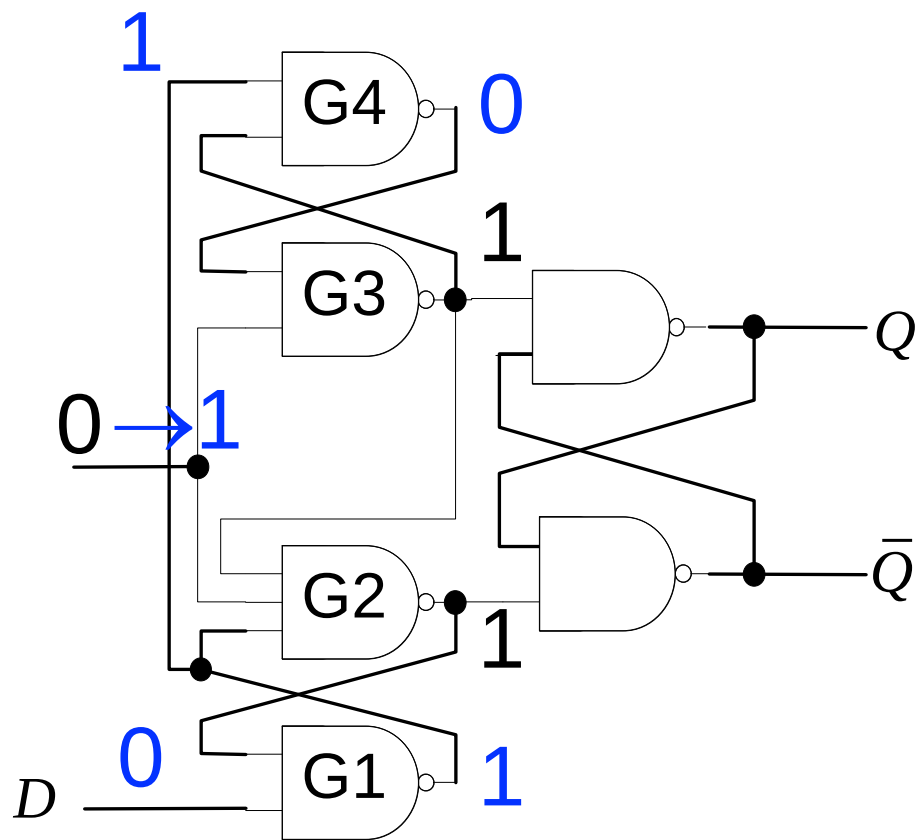


边沿触发DFF

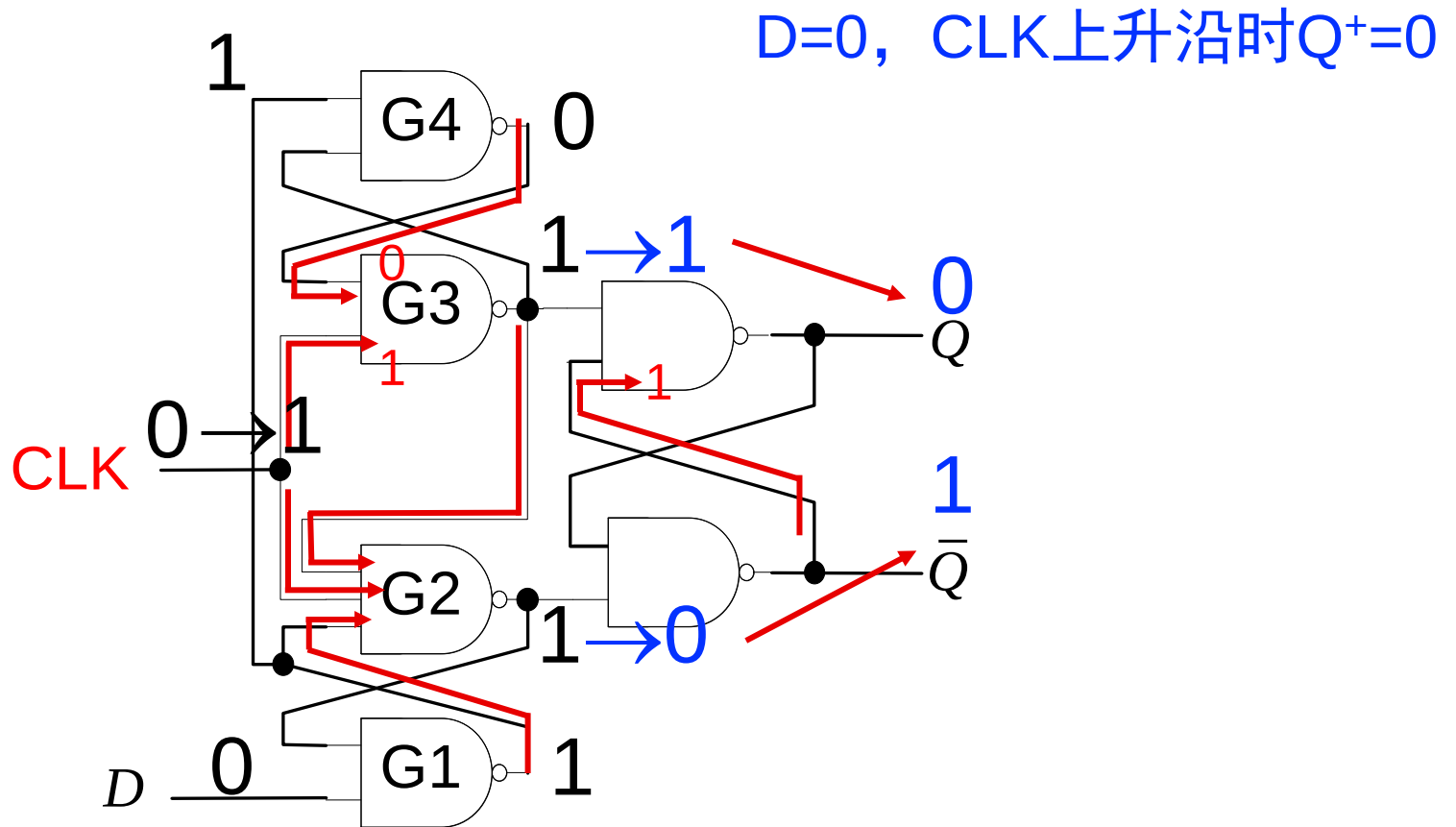


CLK=0时保持

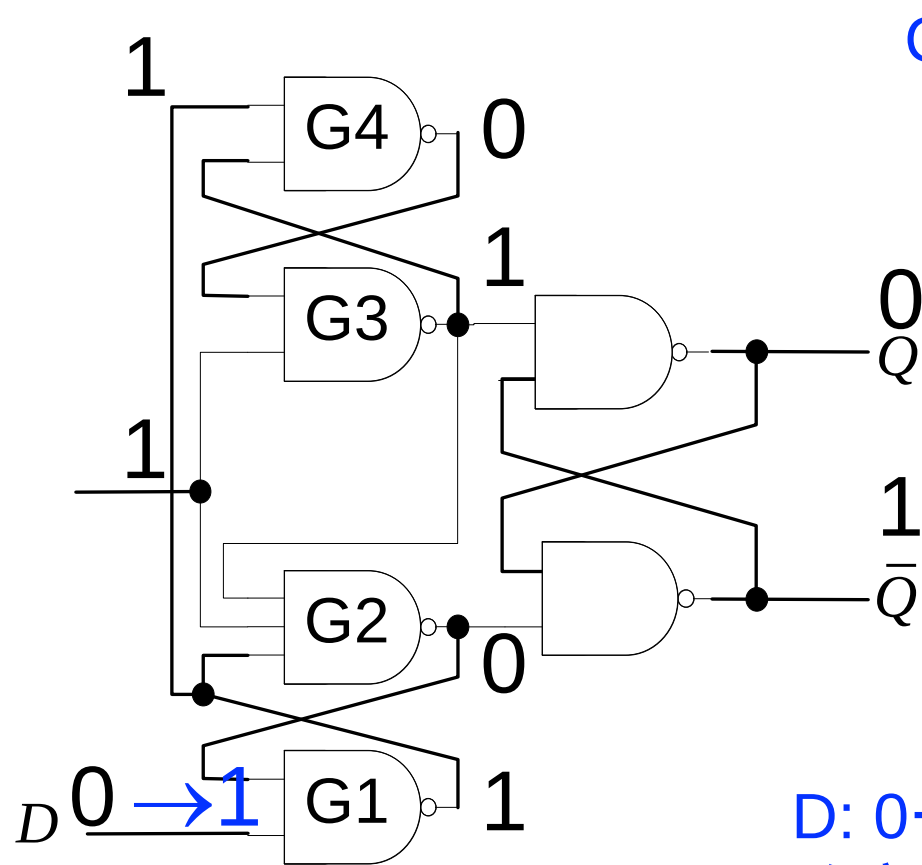
边沿触发DFF



边沿触发DFF



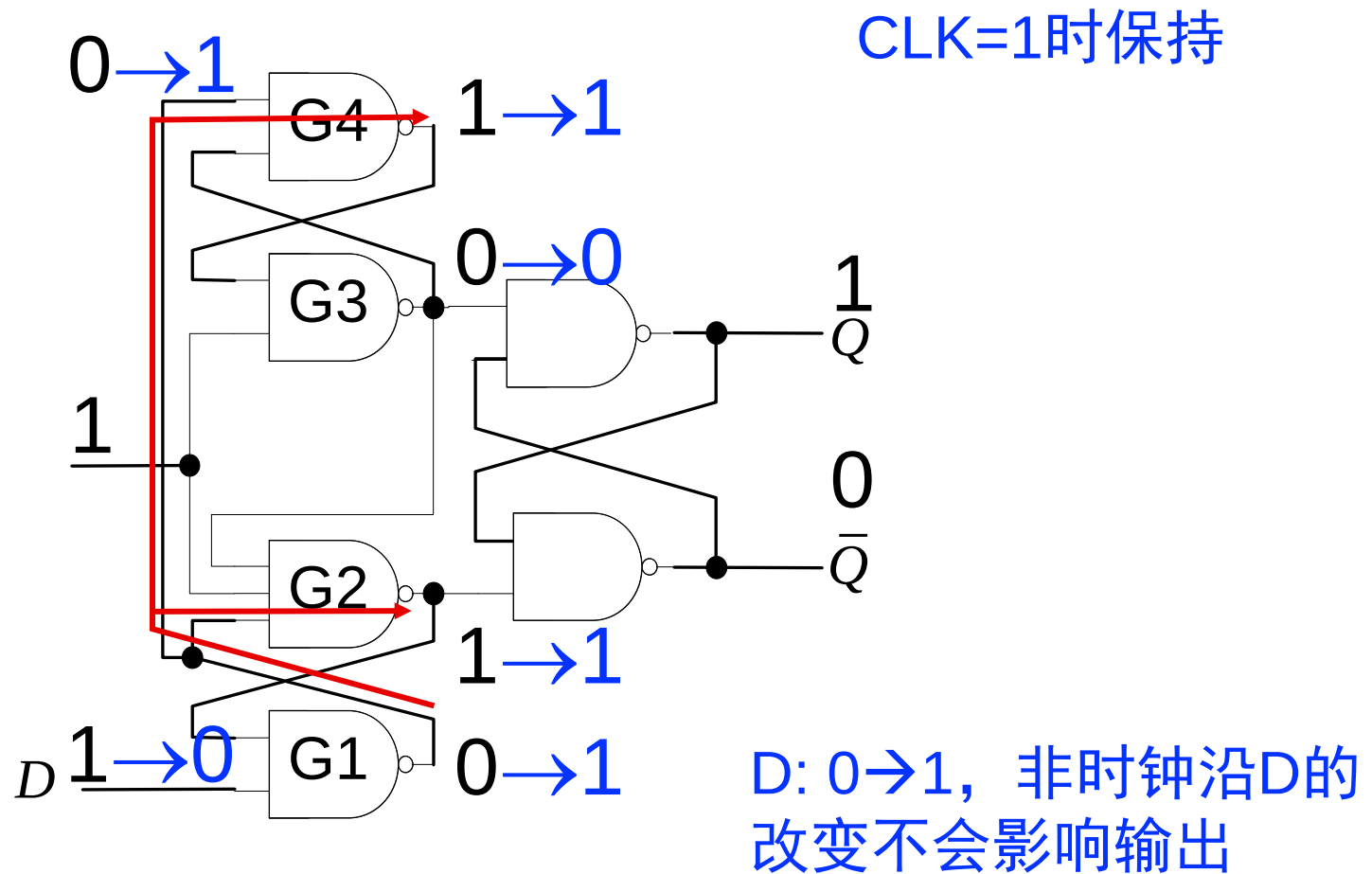
边沿触发DFF



CLK=1时保持

D: 0→1, 非时钟沿D的改变不会影响输出

边沿触发DFF



边沿触发DFF

